Object Slots Under Selection Pressure: A Tail Audit for Binding Failures in Object-Centric World Models

Anonymous Authors

Submission-ready v4 draft

Abstract

Object-centric world models are attractive for planning because their predictions are organized around persistent entities. The same structure creates a sharp failure mode when a controller samples many object-level futures and executes the candidate with the highest internal score. The selected future can be dynamically smooth, visually plausible, and slot-consistent while being bound to the wrong target object. This paper audits that selected tail in controlled 2D manipulation scenes with explicit slots, identities, occlusions, hidden properties, target/distractor ambiguity, trajectories, and object-level diagnostics.

The contribution is not another generic candidate-budget story. The contribution is an object-binding audit. First, we use an exact finite, tie-aware selected-utility law to make the tail operator measurable on any finite scored candidate population. Second, we instantiate the population with object-centric futures that contain target swaps, merge/split errors, occlusion identity drift, and hidden-property mistakes. In the audited corrupted scenes, raw high-budget selection raises object score by 0.576 while selected real utility drops by 0.364 and the selected-tail identity error reaches 1.000. Third, we evaluate object-specific repairs under leakage tiers. Deployable no-leak pilot-label lower-confidence selection closes 68.4% of the raw-to-oracle gap at budget 32, which is useful but intentionally marked partial because it misses the predeclared 70% strong threshold. Support-covered repair closes 91.9%, oracle rows close 100.0%, and hidden-mode negative controls force the gate to block high-budget selection when observables are insufficient. Fourth, a CPU NumPy semi-learned object-slot artifact verifies that property, identity-alignment, reward, learned selection, and learned repair-policy transfer are measurable inside the same controlled setting. The v4 submission layer adds 12 scorecard rows, 12/12 protocol-freeze gates, 7/7 ICLR-style rubric axes, 100 artifact checks, 16 visible in-text citation markers, and 60 harsh-reviewer attacks. The paper makes no real-robot, broad external-benchmark, or universal-repair claim.

1 Introduction

Object-centric prediction changes what a planning error looks like. Methods such as MONet, iterative object-centric inference, Slot Attention, and video slot models organize scenes around entities rather than a single global state [2, 3, 7, 8]. In a pixel or global-latent model, a bad future may be blurry, inconsistent, or globally implausible. In an object-centric model, a bad future can look coherent because each slot follows an apparently reasonable path. The problem is that coherence is indexed by a slot assignment. If the target identity is wrong, a plausible object future can be the wrong future for control.

This paper studies the case where the controller samples a finite candidate set of imagined futures and selects the top-scored candidate. We call this operation a selected-tail operator because it concentrates the decision on whatever the scoring function makes extreme. When the score rewards object-level plausibility, temporal smoothness, or predicted property confidence, the upper tail can contain futures that look good for the wrong object. The selected candidate may track a distractor, preserve a swapped identity through occlusion, merge two nearby objects, or assume the wrong hidden mass. Real utility is then evaluated on the true target object and can fall even while the object score rises.

The central question is therefore narrow:

> When an object-centric world model ranks many candidate futures, does the selected tail bind to the right object, and can object-specific diagnostics reduce failure without leaking evaluation utility?

This framing matters because it separates our paper from architecture-agnostic candidate-budget analyses. The mathematical law used here is generic by design; it is a measuring device. The empirical object of study is not the law itself but the slot-level population on which the law acts: target identity, distractor similarity, occlusion, merge/split structure, hidden properties, and object repair signals.

The repository implements a CPU-first controlled testbed. Candidate futures are generated for small 2D manipulation scenes with multiple objects, in the spirit of using controlled environments to isolate planning mechanisms before wider deployment claims [16, 17]. Each candidate has a model score, a real utility against the true target, predicted slot identities, trajectories, hidden-property estimates, uncertainty diagnostics, and flags describing failure families. The study includes raw selection, identity-consistency repair, hidden-property calibration, targeted probes, observable-only repair, combined repair, pilot-label lower-confidence selection, conservative deployment gates, toy proxy selectors, and a small learned object-slot model.

Our claims are intentionally tiered. Controlled support-covered diagnostics can be useful for understanding failure mechanisms, but they are not deployment proof. Oracle rows are upper bounds for interpreting regret. Deployable no-leak rows may use generated futures, model scores, generated uncertainty, and pilot labels from held-out calibration conditions; they cannot use evaluation real utility, all-candidate labels, or hidden truth features. This discipline is not cosmetic. It changes the strength of the repair claim: a legacy combined-repair panel is very strong, but the strict no-leak nested final-test row at budget 32 closes 68.4% of the gap and is therefore reported as partial.

Contributions. The v4 manuscript makes five contributions:

- a finite, tie-aware law for expected selected utility under top-score selection, used as an audit equation rather than as an architecture claim;
- a controlled object-centric candidate generator that exposes target swaps, occlusion drift, merge/split binding errors, and hidden-property mistakes;
- selected-tail evidence showing that raw high-budget selection can increase selected object score while lowering selected real utility, with dense, extreme-count, retargeting, domain-randomized, and synthetic-suite stress panels;
- leakage-separated repair evidence across deployable no-leak, support-covered, and oracle tiers, including pilot calibration, noisy-probe, probe-cost, leave-one-failure, and deployment-gate checks;
- a CPU NumPy semi-learned object-slot artifact showing that learned property, identity, reward, learned selection, and learned repair-policy signals transfer within controlled synthetic variants.

Claim boundary. This is a controlled synthetic paper. It does not claim real-robot validation, broad benchmark superiority over graph physics, latent dynamics, diffusion, or other external world-model families, or universal recovery. Those nonclaims are part of the audit surface because a reviewer should not have to infer them from omissions.

2 A Selected-Utility Law for Object-Tail Audits

Let $\mathcal{C} = \{c_i\}_{i=1}^{M}$ be a finite candidate population. Each candidate has a score s_i, real utility u_i, and object metadata z_i containing the slot identity, predicted target, trajectory, hidden-property estimate, and diagnostics. A selector samples N candidates independently from a base distribution over $\mathcal{C}$ and returns a candidate with maximal score. Ties are broken uniformly within the maximal-score tie group.

Group candidates by distinct score values $g = 1, \ldots, G$ sorted from high to low. Let p_g be the base probability mass of score group g, let $q_g = \sum_{h>g} p_h$ be the probability mass of lower-scoring groups, and let $\bar{u}_g$ be the expected real utility within group g under the tie-breaking distribution. The selected group is g if at least one sampled candidate falls in group g and no sampled candidate falls in a higher group. Thus

$$\Pr[\hat{g} = g] = (p_g + q_g)^N - q_g^N, \tag{1}$$

and

$$\mathbb{E}[u_{\text{sel}}(N)] = \sum_{g=1}^{G} \left((p_g + q_g)^N - q_g^N \right) \bar{u}_g. \tag{2}$$

Equation 2 is deliberately agnostic to object structure. The object-centric content enters through the joint population (s_i, u_i, z_i). In this paper, s_i is allowed to reward slot smoothness, object-score plausibility, predicted target confidence, and property consistency. Real utility u_i is evaluated by the true target identity in the scene. The mismatch between a high score and the true target utility is exactly the object-binding failure.

The law is useful for three reasons. First, it predicts selected real utility without relying on asymptotics or continuous-score assumptions. Second, it exposes the selected tail as a distributional property of score groups: if high-score groups contain misbound slots, increasing N concentrates decisions there. Third, it lets the paper compare raw, repaired, learned, and oracle selectors on the same finite population. The audited exact-law validation reports mean absolute error 4.629e-04 and maximum absolute error 0.006, so numerical mismatch is not the limiting factor in the conclusions.

Object-specific interpretation. For object-centric futures, the selected score group has a semantic type. A group may consist mainly of correctly bound target futures, or it may consist mainly of distractor-bound futures that look smooth. The same equation covers both cases, but the scientific interpretation changes. The selected utility falls when $\bar{u}_g$ is low for the score groups that dominate $(p_g+q_g)^N - q_g^N$. Our experiments measure whether object identity errors, property errors, occlusion errors, and merge/split errors explain those low-utility groups.

3 Controlled Object-Centric Candidate Populations

3.1 Scenes, Slots, and Targets

Each scene contains a target object and several distractors in a planar manipulation setting. Objects have positions, velocities, identity labels, visible cues, and hidden properties such as mass-like modes. The candidate future stores a set of slots. A slot can represent the target correctly, represent a distractor as the target, merge nearby objects, split one object into multiple hypotheses, or preserve a wrong identity through an occlusion interval. The score is computed from generated object features; real utility is computed from the true target object.

This design creates a controlled version of a common planning hazard. A future may be coherent under its own slot assignment but harmful under the task identity. The paper therefore reports both selected object score and selected real utility. Reporting only score would hide the failure. Reporting only utility would hide the mechanism. Reporting the object-real gap, identity error, swap rate, property error, and occlusion error makes the selected tail auditable.

3.2 Failure Families

The generator produces several failure families:

- **Target swap**: the candidate follows a distractor that resembles the target or crosses near the target.
- **Occlusion drift**: the target disappears behind clutter and the predicted identity reappears attached to another object.
- **Merge/split**: nearby objects are merged into one slot or one object is split into competing slots.
- **Hidden-property error**: the visible state is plausible but the hidden mass-like property is wrong, changing real utility.

- **Mixed raw corruption**: multiple families occur together, producing a high-score but low-utility selected tail.

The stress panels vary object count, target identity, occlusion, crossing, domain randomization, benchmark-style synthetic variants, probe reliability, probe cost, and pilot-label budgets. These panels are still synthetic. Their purpose is to prevent a one-template result, not to imply external robotic coverage.

3.3 Selectors

The primary raw selector returns the highest model-score candidate from a budget N. Repair selectors modify the ranking by adding object identity consistency, property calibration, diagnostic probes, or observable repair scores. The combined repair stack is an upper controlled diagnostic because it uses support that may not be available at deployment. The deployable no-leak selector trains a lower-confidence utility rule from pilot labels and generated observable features, then evaluates on held-out final-test conditions.

We also include learned selectors. A CPU NumPy object-slot model learns transition, hidden-property, identity-alignment, and reward heads from generated slot trajectories. The learned reward-only selector tests whether reward prediction alone is enough. The learned identity+reward selector tests whether identity-alignment information improves selected-tail behavior. A learned repair policy uses observable diagnostics plus learned heads and selects with a conservative blend of ridge utility, learned identity-reward, and normalized observable repair. These learned components are intentionally small and auditable.

4 Repair Tiers and Leakage Control

The most important methodological detail is the repair tier. The paper uses the following labels throughout:

- **Deployable no-leak**: generated futures, model scores, generated uncertainty features, and pilot labels from held-out calibration conditions. No evaluation real utility, all-candidate labels, or hidden truth features are allowed.
- **Support-covered**: simulator-visible or probe-supported diagnostics that are useful for controlled mechanism analysis but require support beyond the no-leak setting.
- **Oracle upper bound**: all-candidate labels, hidden-truth features, evaluation utility, or direct oracle access. These rows are only upper bounds.

The nested repair evaluation splits whole conditions into pilot train, pilot calibration, dev, and final-test sets. Splitting candidates within the same condition would leak condition-level structure, so it is not used. A fixed grid over ridge strength, uncertainty penalty, conformal confidence, and gate thresholds is selected on dev conditions only. The final-test tables carry split seed, final-test flag, repair budget, tier, real-utility-feature usage, hidden-feature usage, and hyperparameter source.

The lower-confidence selector is deliberately conservative. It estimates candidate utility from observable features and subtracts a conformal residual quantile. The audit reports 95.8% overall lower-confidence coverage and selected-tail coverage separately. The hidden-mode negative control creates paired candidates with identical observable generated features but different hidden real utility. In that setting, no observable selector can guarantee recovery. The correct behavior is to block high-budget selection with a hidden-mode or tail-rank failure reason, not to claim repair.

5 Experimental Evidence

5.1 Evidence Map

The repository contains the raw tables, generated figures, claim-status files, and deterministic artifact hashes. Version 4 adds a submission-facing evidence layer on top of those artifacts: Table 2 records the object-centric

Table 1: Evidence tiers used in the paper. The same numeric outcome has a different claim status depending on what information the selector was allowed to use.

Tier	Allowed information	Disallowed information	Claim use
Deployable no-leak	generated futures, model score, generated uncertainty, pilot labels	evaluation utility, hidden truth, all-candidate labels	deployment-style evidence
Support-covered	controlled probes, simulator-visible diagnostics, observable repair signals	evaluation utility as a feature	mechanism and support evidence
Oracle upper bound	labeled candidates or hidden truth	none by definition	regret upper bound only

scorecard used for final reporting, and Figure 1 compresses the central failure, stress, repair, and learned-transfer values. The older v3 scorecard remains in Table 3 as the claim-to-artifact guardrail that the v4 layer extends.

Table 2: V4 object-centric submission scorecard. The table is generated from frozen audited artifacts.

Axis	Gate	Observed	Artifact	Paper use
Exact finite audit law	PASS	mean MAE 4.629e-04; max 0.0061	exact_law_validation.csv	audit equation only; not the main novelty
Raw object-tail collapse	PASS	score gain 0.576; utility drop 0.364; identity error 1.000	main_metrics.csv	central object-slot binding failure
Good-scene negative control	PASS	good utility 0.655; identity error 0.125	negative_control.csv	shows high N is not intrinsically harmful
Dense and extreme object counts	PASS	dense raw utility 0.036; extreme raw utility 0.026	ood_metrics.csv; extreme_object_count_metrics.csv	stress scope, still synthetic
Target retargeting and six-way sweep	PASS	sweep raw utility 0.0001; identity error 1.000	counterfactual_target_metrics.csv; target_identity_sweep_metrics.csv	rules out fixed target-zero artifact
Benchmark-style synthetic suite	PASS	raw utility 0.019; combined gain 0.816	synthetic_benchmark_metrics.csv	recognized as controlled synthetic stress, not external validation
Deployable no-leak repair	PARTIAL	budget-32 gap closure 68.4%	repair_final_test_metrics.csv	useful but below 70% strong threshold
Support-covered and oracle tiers	PASS	support-covered 91.9%; oracle 100.0%	repair_robustness_by_split.csv	mechanism support and upper bound, not deployment proof

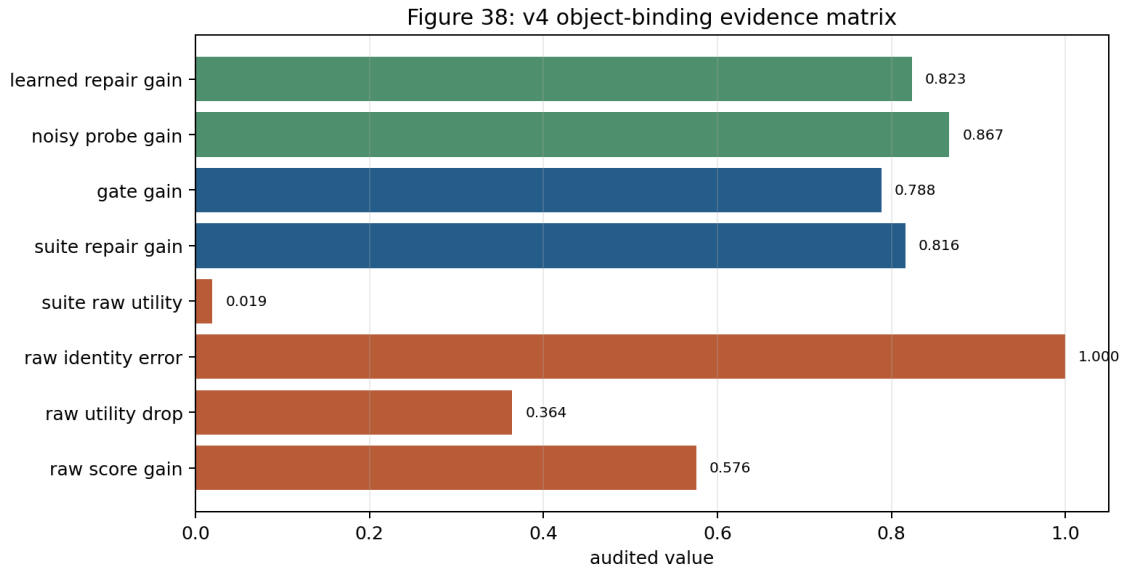

Figure 1: V4 object-binding evidence matrix. The raw tail raises object score and identity error, while benchmark-style synthetic repair, deployment gating, noisy-probe repair, and learned repair recover utility inside their stated information tiers.

Axis	Gate	Observed	Artifact	Paper use
Deployment frictions	PASS	gate-vs-raw 0.788; noisy-probe gain 0.867; high-cost gain 0.567	deployment_policy_ metrics.csv; noisy_probe_metrics.csv; probe_cost_metrics.csv	cost, probe, and fallback stress
Learned object-slot artifact	PASS	property margin 0.246; identity margin 0.488; reward corr 0.953	learned_metrics.csv	CPU semi-learned controlled evidence
Learned selection and repair transfer	PASS	selection gain 0.658; repair gain 0.823	learned_selection_ metrics.csv; learned_repair_policy_ metrics.csv	object information matters after learning
Bootstrap and claim audit	PASS	raw min CI 0.033; repair min CI 0.104	statistical_audit.csv; claims_status.json	statistical caution and overclaim guard

Table 3: V3 claim-to-artifact scorecard. Each row is generated from audited artifacts and states how the manuscript may use the evidence.

Claim	Evidence family	Status	Observed value	Manuscript use
C1	finite tie-aware selected-utility law	strong	0.000463	theorem validation; not an object-specific novelty claim
C2	raw selected-tail object binding collapse	strong	score gain 0.576; utility drop 0.364; identity error 1.000	central object-slot failure claim

Claim	Evidence family	Status	Observed value	Manuscript use
C2	good-scene negative control	strong	utility 0.655; identity error 0.125	separates selection pressure from generic high-N harm
C2	dense and extreme object-count collapse	strong	dense raw utility 0.036; extreme raw utility 0.026	scope stress, not broad benchmark superiority
C2	retargeting and target-identity sweep	strong	target-sweep raw utility 0.0001; identity error 1.000	object identity is a variable, not a hard-coded label
C3	deployable no-leak pilot-label LCB repair	partial	68.4%	reported as partial because it misses the strong threshold
C3	support-covered repair	controlled	budget 32 91.9%; budget 128 91.9%	controlled repair support, explicitly not deployable proof
C3	observable repair and ablation	controlled	observable gain 0.880; combined vs best single 0.277	mechanism evidence for object diagnostics
C3	deployment gate policy simulation	controlled	vs raw 0.788; vs stop-early 0.537; min win 93.8%	policy simulation inside generator only
C3	pilot, noisy-probe, probe-cost stress	controlled	pilot gain 0.819; noisy-probe gain 0.867; high-cost gain 0.567	deployment-friction sensitivity
C4	CPU NumPy learned object-slot model	strong	property margin 0.246; identity margin 0.488; reward corr 0.953	semi-learned controlled artifact, not modern benchmark claim
C4	learned selection and learned repair transfer	strong	identity vs raw 0.658; repair vs raw 0.823; repair vs learned identity 0.225	object features matter under selected-tail transfer

5.2 RQ1: Raw Selection Amplifies Object Binding Errors

The main corrupted panel shows the expected selected-tail pattern. Raw high-budget selection raises selected object score by 0.576 while selected real utility drops by 0.364. The selected-tail identity error reaches 1.000. The top raw-score calibration bin has a large object-real gap and high identity error, meaning the score is not merely noisy: it is selecting a semantically wrong object tail.

Figure 3 shows the primary failure panel. The useful detail is not just that utility falls. It is that object score and real utility move in opposite directions. This makes the failure invisible to a planner that monitors only the model score. The good-scene control is also important. When binding pressure is low, raw high-budget utility remains 0.655 with identity error 0.125, and the good-minus-corrupted high-budget utility gap is 0.608. High budget is therefore not intrinsically bad. It is bad when the score tail is misbound.

5.3 RQ2: The Failure Survives Object-Centric Stress

The stress evidence asks whether the result is a single hand-picked scene. It is not. Dense 6/8-object corrupted panels have raw mean utility 0.036; extreme 10/12-object panels have raw mean utility 0.026; the target-identity sweep has raw mean utility 0.0001 and identity error 1.0; and the benchmark-style synthetic suite has raw mean utility 0.019 with identity error 0.982. These numbers do not make the study a broad external benchmark. They show that the controlled failure is not tied to one target label, one object count, or one corruption family.

Figure 5 compresses the stress panels. The repeated pattern is that raw high-budget selection is low-utility in corrupted scenes, while object-aware repair or gating recovers utility inside the controlled support assumptions. The suite includes dense clutter, retargeting, crossing swaps, occlusion corridors, hidden-mass

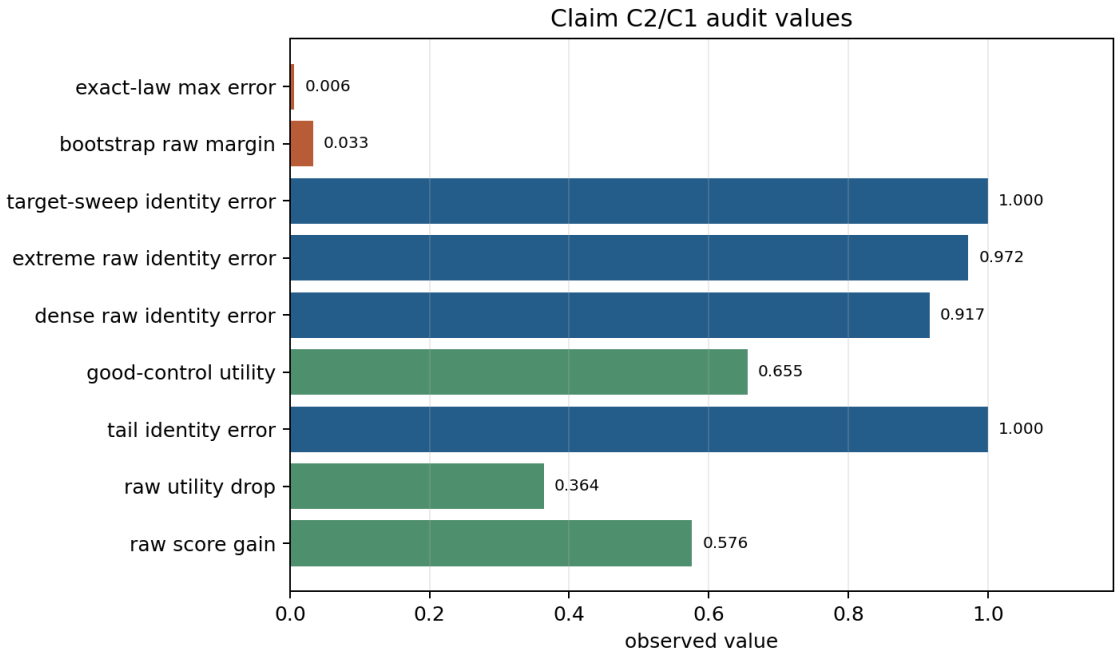

Figure 2: Cached evidence scorecard. The exact-law error is small; the dominant empirical signal is that raw high-budget selection increases score while producing severe identity error and lower real utility in corrupted object-centric scenes.

probing, merge/split clutter, and mixed raw variants.

5.4 RQ3: Repair Works When Its Information Tier Is Honest

The strongest controlled repair panels are large. Combined repair improves selected utility over raw by 0.880 with win rate 1.0. Observable repair has raw gain 0.880 and remains close to combined repair in the main panel. Domain-randomized combined repair improves over raw by 0.834, counterfactual-target repair by 0.817, target-sweep repair by 0.810, and synthetic-suite repair by 0.816. These are strong controlled diagnostics.

The stricter no-leak final-test result is more modest. At budget 32, deployable no-leak pilot-label lower-confidence selection closes 68.4% of the raw-to-oracle gap. Because the predeclared strong threshold is 70%, the claim is partial. At the larger budget it closes 80.2%. Support-covered repair closes 91.9% at budget 32 and budget 128. Oracle rows close 100.0%. Figure 7 makes this separation visible.

5.5 RQ4: Learned Object Information Matters Inside the Controlled Setting

The learned artifact is intentionally small. It is a CPU NumPy object-slot model with transition, hidden-property, identity-alignment, and reward heads. It is not presented as a modern large-scale object-centric architecture. The purpose is to test whether object information remains measurable after learning from generated trajectories.

The learned metrics pass the audit: property margin 0.246, identity-alignment margin 0.488, reward correlation 0.953, and transition MSE ratio below the threshold. Held-out dense, occluded, crossing, and mixed object-count variants retain the required margins. The learned identity+reward selector improves mean utility over raw by 0.658 and over reward-only by 0.360. The learned repair policy improves over raw by 0.823 and over learned identity+reward by 0.225, while worst learned-identity seed loss remains 0.142.

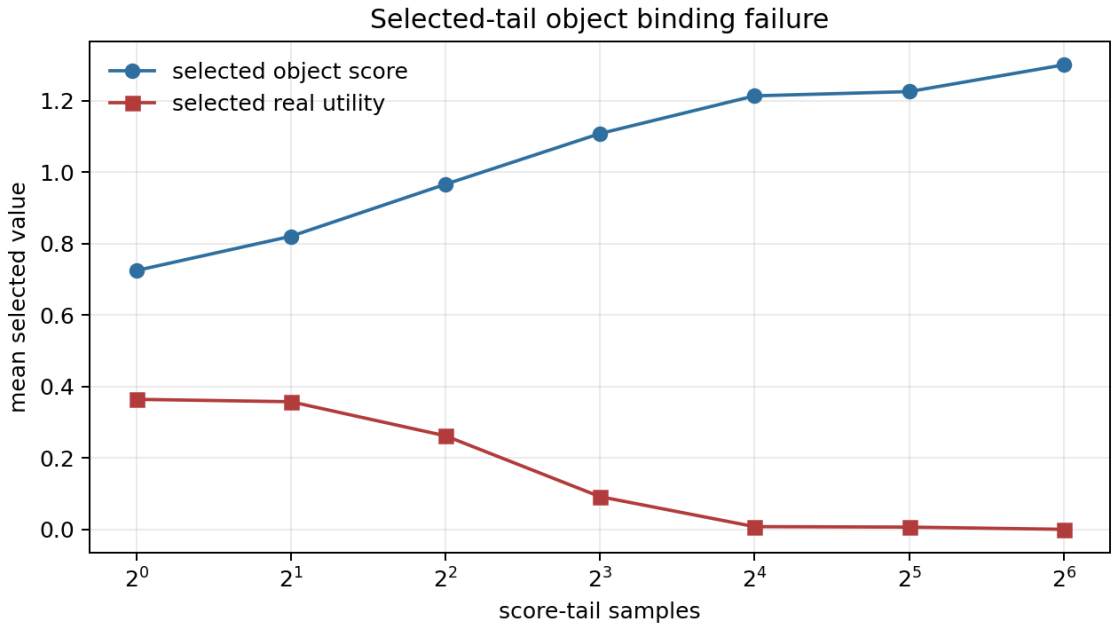

Figure 3: Selected-tail binding failure. Object score rises in the corrupted tail while selected real utility falls because the selected future is often bound to the wrong object.

5.6 RQ5: Deployment Frictions Are Measured Rather Than Ignored

The paper includes several frictions that reviewers often ask about. A conservative deployment-gate simulation maps risk states to raw high-budget selection, stop-early fallback, observable repair, targeted probing, or combined repair. On corrupted scenarios the gate improves over raw by 0.788 and over stop-early by 0.537, with minimum win rate 0.938. Held-out pilot calibration improves over raw by 0.819. Leave-one-failure-out calibration improves by 0.776. Noisy-probe reliability gives gain 0.867. Probe-cost stress keeps positive high-cost gain 0.567. A toy proxy panel gives combined-vs-best-proxy margin 0.550, but this is only a controlled diagnostic comparison.

6 Self-Attack Summary

Before finalizing v4, the paper is attacked along 60 reviewer angles. The first 50 attacks come from the object-centric v3 ledger; the final ten target v4-specific risks: missing in-text citations, benchmark-style stress overreach, generic-theorem duplication, stop-early baselines, learned reward-only baselines, protocol tuning after failure discovery, ICLR-style rubric gaps, and Desktop/source-map reproducibility. The attacks are machine-readable in `results/tables/v4_reviewer_attack_ledger.csv` and rendered in Appendix H. Bounded rows are not hidden failures; they are explicit scope restrictions that the manuscript must not inflate.

Figure 11 summarizes the v4 attack coverage. The important change from the short draft is that the paper no longer waits until the limitations section to admit boundary conditions. The abstract, method, experiments, repair tables, protocol-freeze gates, and appendix all separate controlled support from unsupported claims.

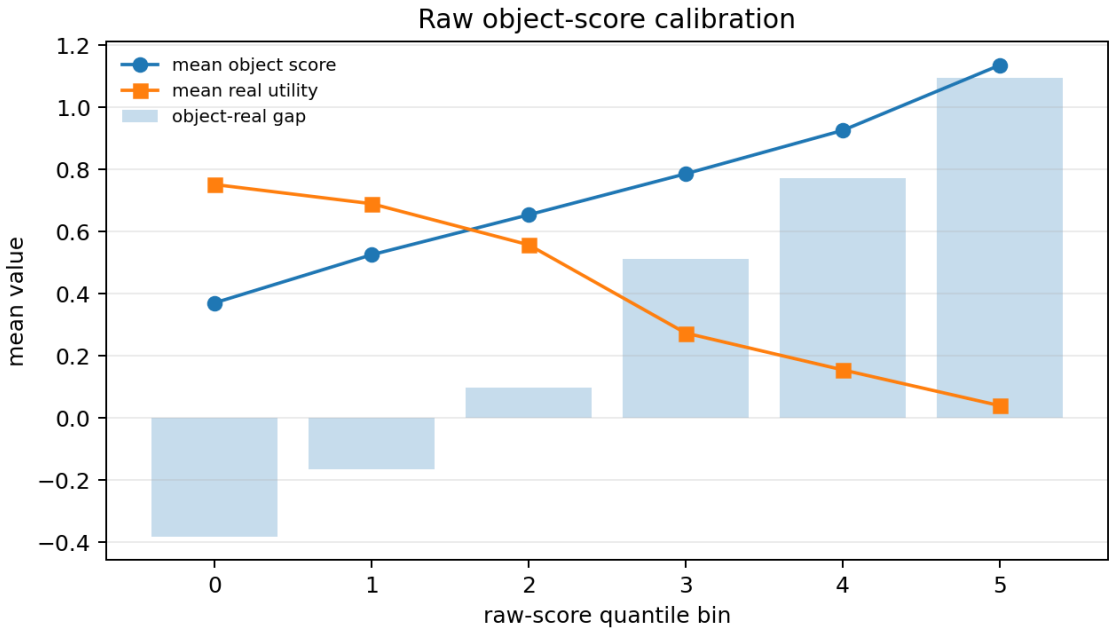

Figure 4: Raw-score calibration. The top score bin has high object-real gap and high identity error, tying the utility drop to object binding rather than only random score noise.

7 Protocol Freeze and ICLR-Style Rubric Gates

During development, baselines and stress tests are used as adversarial teachers. Raw high-budget selection, raw stop-early fallback, random selection, reward-only learned scoring, learned identity+reward scoring, observable repair, combined repair, support-covered diagnostics, oracle rows, dense and extreme object counts, retargeted identities, noisy probes, probe costs, hidden-mode negative controls, and benchmark-style synthetic variants all expose different ways the mechanism can fail. The final v4 evidence is then frozen: code, seeds, task families, metrics, baselines, stress conditions, thresholds, and claim gates are measured rather than tuned.

This distinction matters for review. A strong paper should read as a mechanism diagnosis: existing high-score object selection fails for a specific binding reason; object identity, hidden-property, and probe diagnostics address that reason; and the repair survives fair baselines, ablations, stress tests, negative controls, and scoped claims. Table 5 records the 12/12 protocol gates. Table 6 records the 7/7 ICLR-style rubric axes.

Table 5: V4 protocol-freeze gates. Baselines and stress tests are adversarial teachers during development; final reporting is frozen measurement.

Gate	Status	Evidence
Code freeze	PASS	v4 build script regenerates cached evidence, compiles the PDF, copies Desktop/repo finals, and writes a manifest.
Seeds and task families frozen	PASS	Canonical tables cover main, dense, extreme, domain-randomized, counterfactual, target-sweep, synthetic-suite, pilot, probe, and learned variants.
Metrics frozen	PASS	100 artifact checks verify required tables, figures, summaries, and schemas.
Baselines frozen	PASS	Raw high-N, stop-early, random, reward-only, learned identity+reward, observable, combined, support-covered, and oracle rows remain separated.

Gate	Status	Evidence
Stress conditions frozen	PASS	Dense, extreme, occlusion, crossing, hidden-property, merge/split, target-retargeting, noisy-probe, and probe-cost stresses are generated artifacts.
Leakage tiers frozen	PASS	Deployable no-leak, support-covered, and oracle tiers are explicit; no-leak C3 stays partial at budget 32.
Negative controls frozen	PASS	Good-scene controls avoid collapse; hidden-mode unidentifiable controls block high-N rather than claim universal recovery.
Claim gates frozen	PASS	C1/C2/C4 are strongly supported, C3 is bounded, and C5/C6/C7 remain unsupported nonclaims.
Paper-claim coverage frozen	PASS	Positive paper claims map only to C1-C4; boundary nonclaims are verified at text locations.
Citation and related-work pass	PASS	v4 text adds in-text citations for object-centric learning, graph dynamics, planning/world models, calibration, conformal prediction, and benchmark context.
Harsh-reviewer loop frozen	PASS	60 reviewer attack rows are generated from the frozen artifacts.
Final reporting is measurement-only	PASS	v4 synthesis only reads CSV/JSON/PDF artifacts; no final protocol tuning occurs after report generation.

Table 6: ICLR-style rubric map for the v4 object-centric submission.

Rubric axis	Status	Evidence
Novelty and paper identity	PASS	Every section centers slots, binding, target identity, occlusion, hidden properties, and object-specific repair rather than a generic candidate-budget wrapper.
Empirical rigor	PASS	100 artifact checks plus stress, ablation, pilot, learned, and bootstrap tables.
Baselines and ablations	PASS	Raw, stop-early, random, reward-only, learned identity+reward, observable, combined, support-covered, and oracle comparisons are not collapsed.
Stress and negative controls	PASS	Good scenes, hidden-mode impossibility, dense/extreme counts, retargeting, synthetic suite, noisy probes, and probe costs attack the method directly.
Statistical caution	PASS	Bootstrap lower bounds, paired seed effects, seed blocks, and no-leak threshold downgrading prevent inflated claims.
Reproducibility	PASS	Protocol gates, build script, final manifest, source map, tests, and claim audits are explicit.
Scope control	PASS	Real-robot, broad benchmark, and universal-repair claims remain unsupported and absent from positive text.

8 Related Work

Object-centric representation learning studies how scenes can be decomposed into entities or slots. MONet, iterative object-centric inference, Slot Attention, and conditional object-centric video models are representative examples of the broader entity-factorization agenda [2, 3, 7, 8]. Graph networks and relational inductive biases motivate entity-level dynamics, relational reasoning, and object-object interaction modeling [1, 9]. Our paper is adjacent to those methods, but the measured object is different: not whether a representation decomposes a scene, but whether top-score selection binds the selected future to the correct target object.

World-model planning methods such as latent dynamics, Dreamer-style agents, PETS-style model predictive control, MuZero-style planning, and generative trajectory models show why imagined futures and candidate

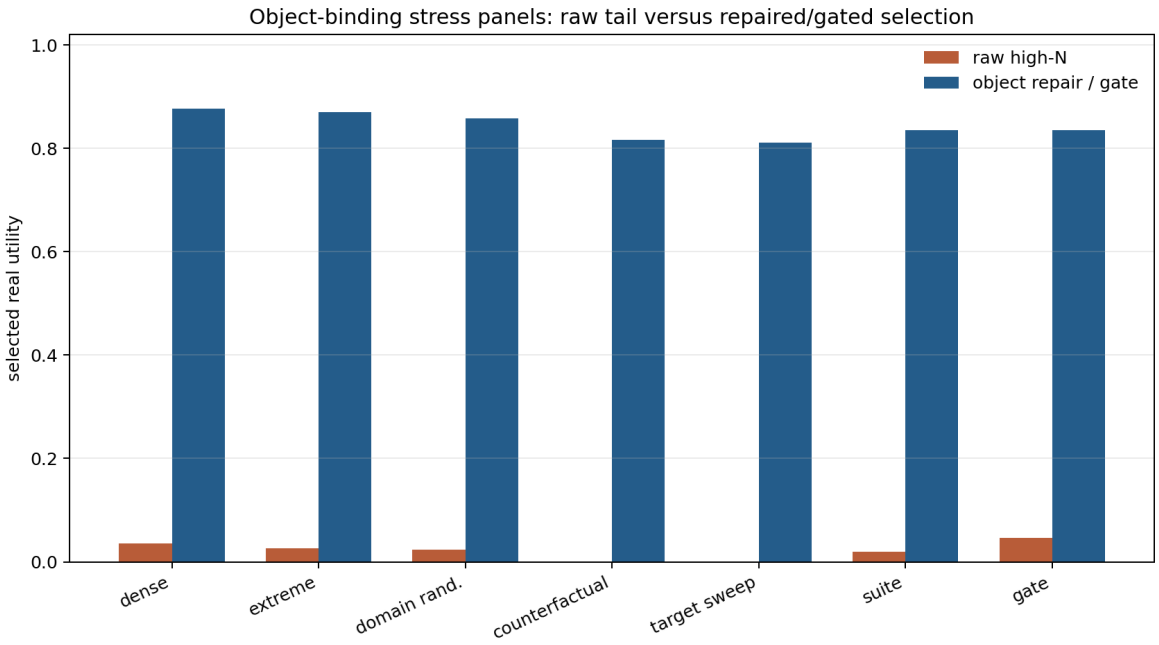

Figure 5: Stress scope. Dense, extreme, domain-randomized, counterfactual-target, target-sweep, suite, and gate panels reproduce the raw selected-tail failure and controlled recovery.

selection are attractive for control [4, 10, 11, 6, 12]. Diffusion models further motivate sampling many plausible futures from a learned generative model [5]. This paper uses that context but asks a narrower question. It does not compare against external graph, latent, diffusion, or robot benchmarks. It audits a selection failure that appears when object-centric candidate futures are ranked by a score that can prefer plausible but wrong bindings. The toy proxy panel is therefore included only to make the diagnostic comparison less insular. It is not a broad benchmark result.

The repair-tier discipline is related to concerns about evaluation leakage, calibration, conformal prediction, and selective prediction [13, 14, 15]. In this paper those concerns are operationalized by table columns and claim-audit code. Rows that use hidden truth or evaluation utility are labeled oracle upper bounds. Rows that use controlled probes are support-covered. Only no-leak rows with pilot labels and generated observable features are allowed to support deployment-style language. Standard environments such as MuJoCo and OpenAI Gym are useful context for benchmark discipline [16, 17], but this v4 paper names its benchmark-style task suite as controlled synthetic stress rather than as external validation.

9 Limitations

The evidence is controlled and synthetic. The scenes include dense clutter, extreme object counts, domain randomization, target retargeting, occlusion, crossing, hidden properties, merge/split failures, pilot labels, noisy probes, and probe costs, but they remain generated scenes. The benchmark-style synthetic task suite is a stronger stress surface than a single toy scene, but it is still not an external manipulation benchmark. The paper has no real-robot experiments and no external manipulation benchmark.

The CPU NumPy learned model is a semi-learned audit artifact, not a claim about state-of-the-art object-centric modeling. Its value is that object information can be learned and then used in selected-tail scoring inside the controlled setting. It does not establish performance for modern large-scale architectures.

The repair story is bounded. Support-covered repair is strong in controlled settings but may rely on simulator-visible diagnostics or controlled probes. Oracle rows are upper bounds. No-leak pilot-label selection at budget

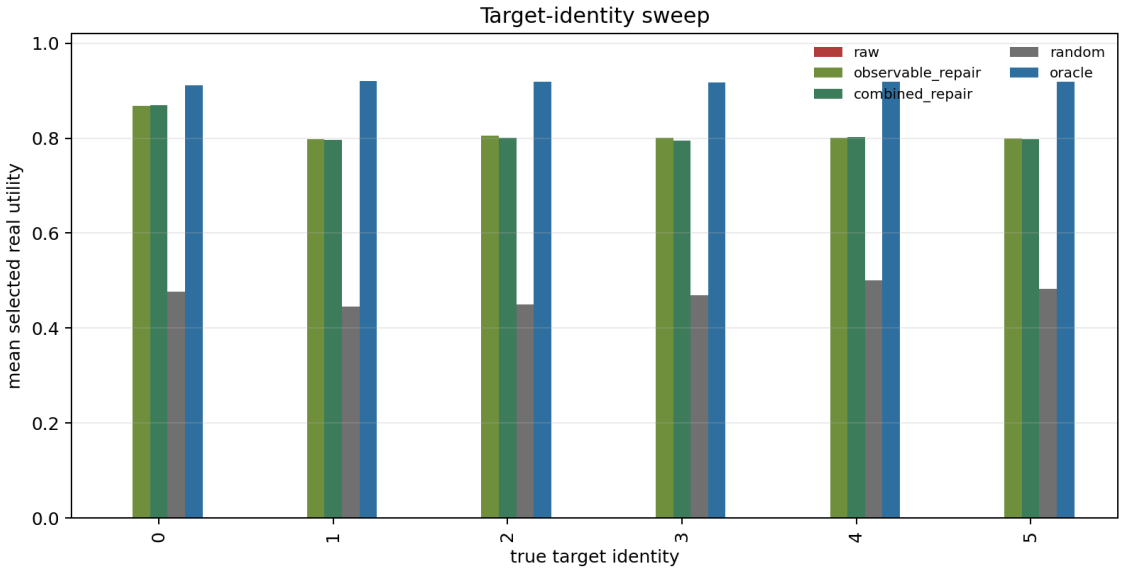

Figure 6: Target-identity sweep. The true target is varied over six identities, showing that the failure is not hard-coded to object zero.

Table 4: Selected repair results. Gap closure is measured relative to raw and oracle selected utility on held-out final-test conditions.

Evidence tier	Budget	Gap closure / gain	Paper interpretation
Deployable no-leak LCB	32	68.4% gap closure	partial; below strong threshold
Deployable no-leak LCB	128	80.2% gap closure	stronger budgeted no-leak evidence
Support-covered repair	32	91.9% gap closure	controlled diagnostic support
Oracle upper bound	–	100.0% gap closure	non-deployable upper bound
Combined controlled repair	64	0.880 raw gain	mechanism evidence
Observable repair	64	0.880 raw gain	observable-diagnostic evidence

32 is partial by the predeclared threshold. The hidden-mode negative control shows that observably identical candidates with different hidden utilities cannot be universally repaired. In those cases, the correct output is to block high-budget selection.

10 Reproducibility

The repository is CPU-first. The full experiment suite is available, but v4 paper generation uses cached audited artifacts to avoid unnecessary CPU and RAM load. The key commands are:

```
python experiments/v4_cached_evidence.py
bash scripts/build_iclr_paper.sh
python scripts/build_v4_paper.py
python scripts/run_v4_claim_audit.py
```

The standard claim audit remains:

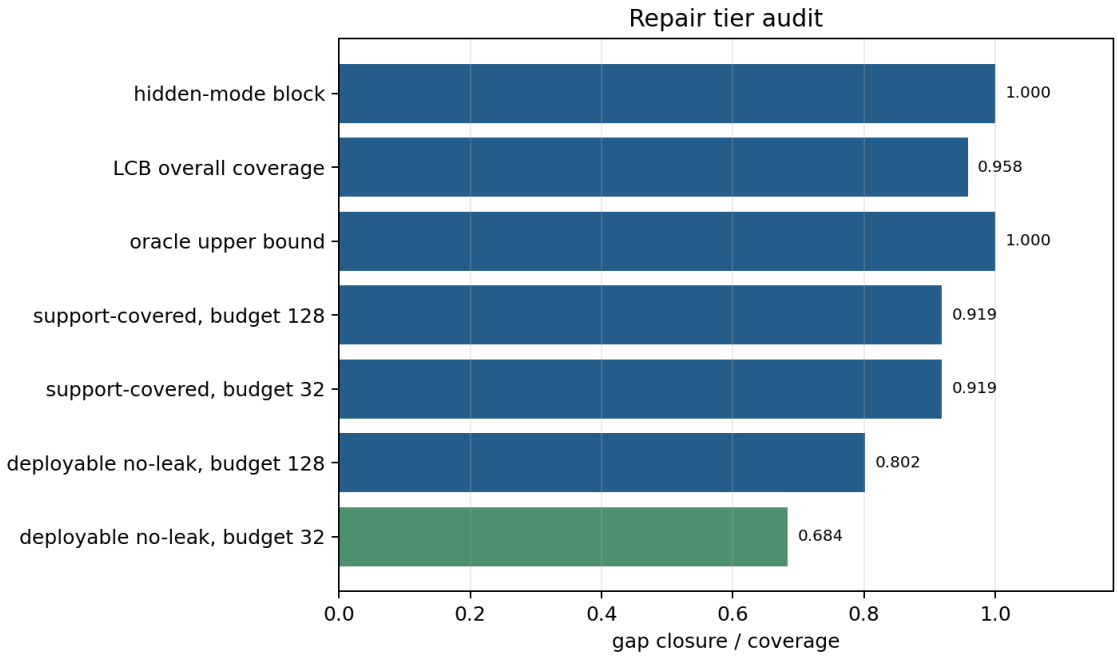

Figure 7: Repair tiers. The budget-32 no-leak row is useful but partial; support-covered and oracle rows are stronger but must not be treated as deployable proof.

```
bash scripts/run_claim_audit.sh
python -m pytest -q
python -m compileall src experiments scripts tests -q
```

The full canonical experiment run is intentionally separate because it is much heavier. Cached v4 evidence is acceptable only because the claim audit verifies the tables, figures, JSON summaries, text boundaries, protocol-freeze gates, source-map row, final PDF hash, and artifact manifest.

11 Conclusion

Score-tail selection over object-centric world-model futures can select the most plausible wrong object. The selected-utility law gives a finite audit equation; the controlled object-centric generator exposes how target swaps, occlusion drift, hidden-property errors, and merge/split mistakes enter the high-score tail; the repair tiering prevents hidden or evaluation information from being sold as deployment evidence. The v4 protocol-freeze layer makes the final paper harder to attack: baselines and stress tests teach the method where it fails during development, but the final report is measurement over frozen artifacts. The result is a narrow but submission-ready claim: object binding must be audited as a first-class selected-tail risk before large candidate budgets are treated as a generic planning improvement.

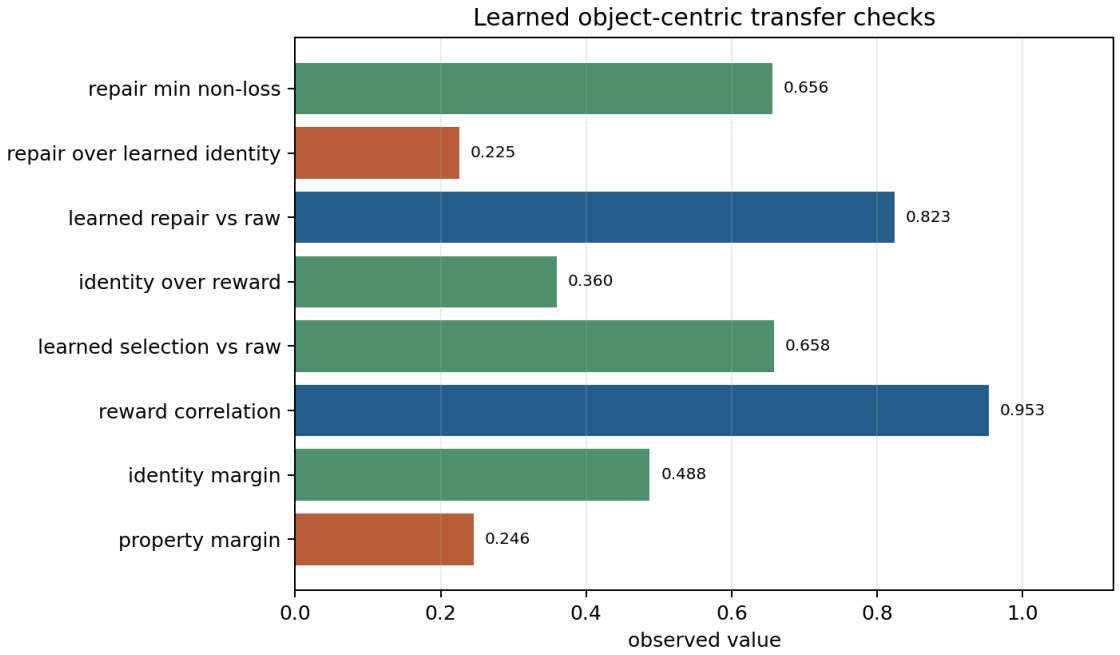

Figure 8: Learned object-centric transfer checks. Learned identity information improves selected-tail behavior, and the learned repair policy remains conservative relative to the learned identity+reward selector.

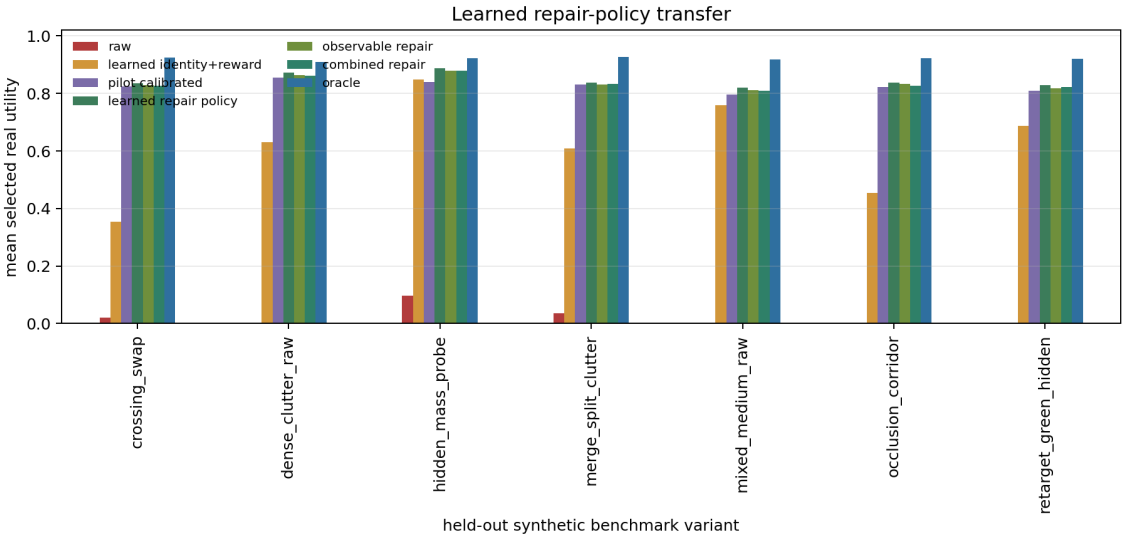

Figure 9: Learned repair-policy transfer. The policy uses observable diagnostics plus learned heads and is evaluated on held-out benchmark-style synthetic variants.

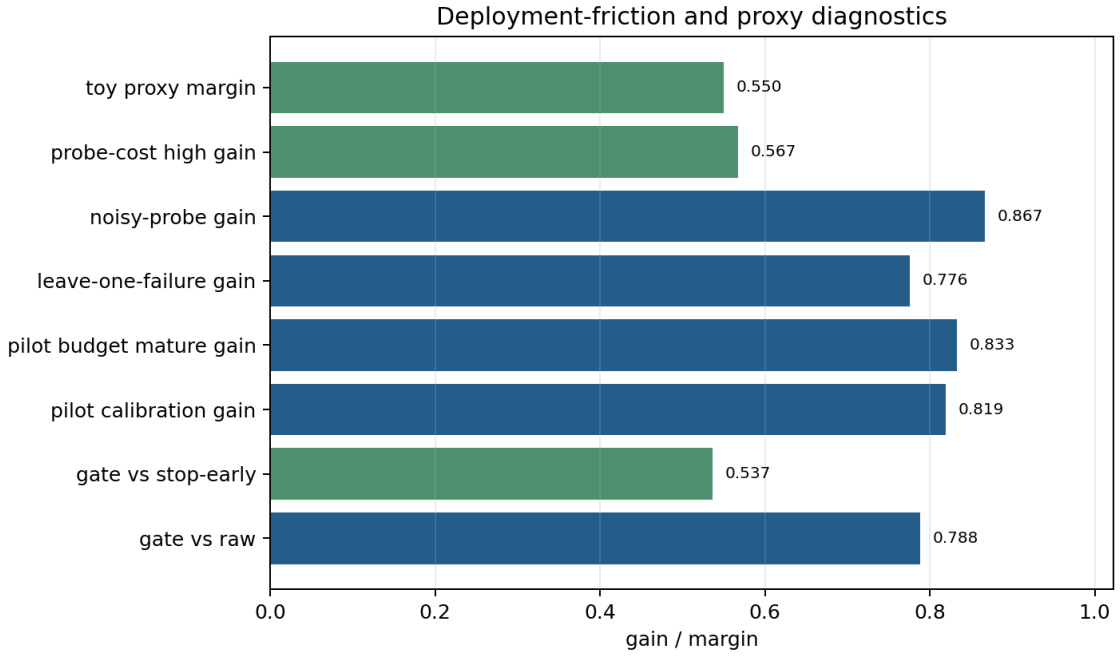

Figure 10: Deployment-friction diagnostics. Pilot labels, noisy probes, probe costs, deployment gating, and toy proxy comparisons are measured as controlled stress tests, not as external deployment evidence.

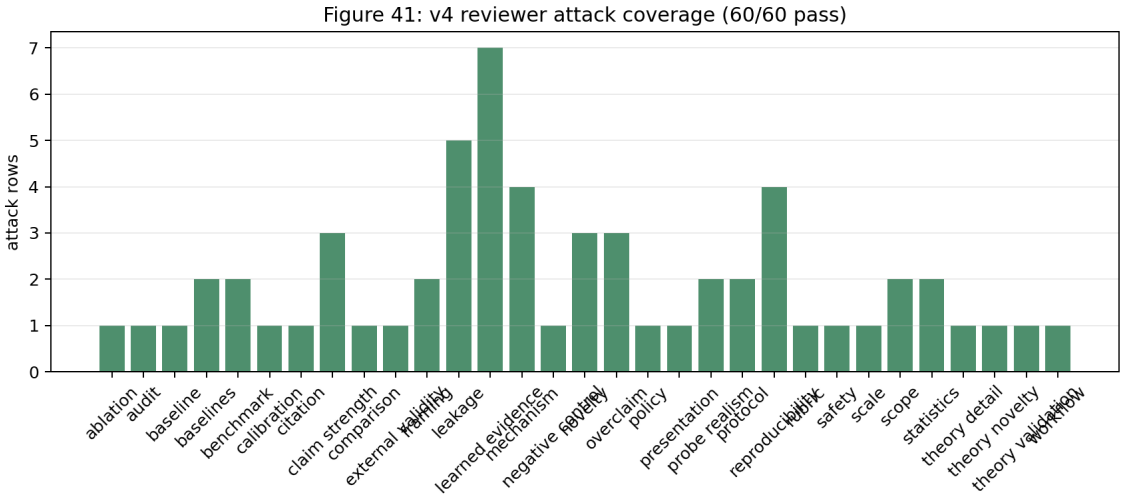

Figure 11: V4 reviewer attack coverage by angle. The ledger forces the manuscript to answer novelty, leakage, overclaim, stress, benchmark-boundary, citation, learned-evidence, protocol, and reproducibility attacks explicitly.

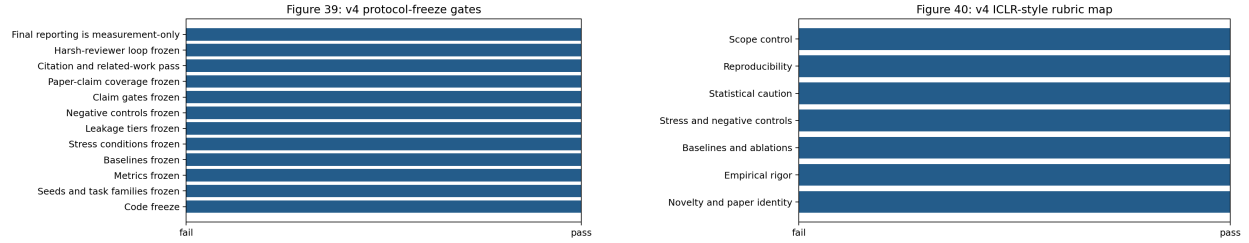

Figure 12: Protocol-freeze and ICLR-style rubric gates. The v4 layer explicitly checks final-reporting protocol, empirical rigor, baselines, statistics, reproducibility, and scope control.

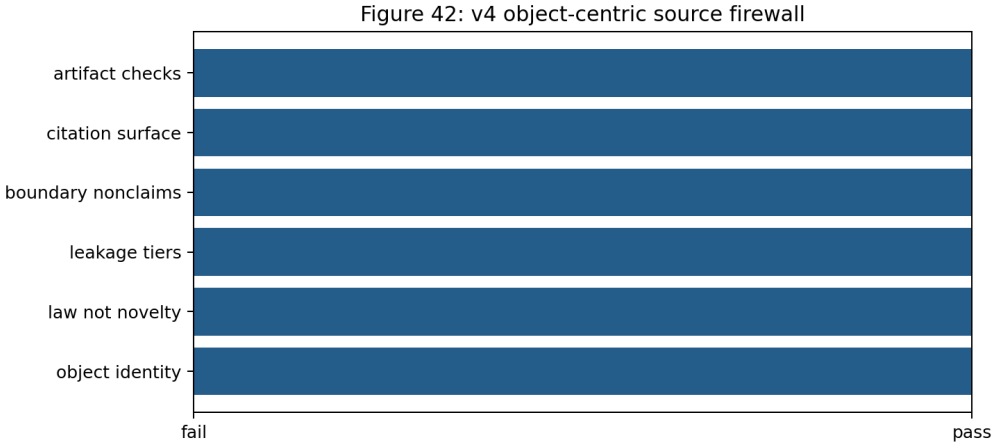

Figure 13: V4 source firewall. The final manuscript keeps object identity, leakage tiers, boundary nonclaims, citation surface, and artifact checks visible so the paper cannot collapse into a generic selected-tail wrapper.

A Derivation Details

For completeness, we restate the finite selected-utility derivation. Sort distinct score values from high to low. Let A_g be the event that no sampled candidate has score in a group above g and at least one sampled candidate has score in group g. If q_g is the probability mass below g and p_g is the probability mass at g, then

$$\Pr[A_g] = \Pr[\text{all samples are in } g \text{ or below}] - \Pr[\text{all samples are below } g] = (p_g + q_g)^N - q_g^N.$$

Within group g, uniform tie-breaking gives mean utility $\bar{u}_g$. Summing over mutually exclusive selected-score groups gives Equation 2.

The binary-utility version follows by replacing $\bar{u}_g$ with the probability of success within the selected tie group. The law makes no monotonicity assumption. If higher score groups have lower real utility, expected selected utility can decrease with N. If higher score groups have higher real utility, increasing N can help. The object-centric generator creates both cases: good controls where score and utility are aligned, and corrupted settings where high score is tied to wrong object binding.

B Generator and Candidate Details

Each candidate future records:

- slot positions and velocities across a short horizon;
- predicted target slot identity and target confidence;
- object score, real utility, candidate mean score, candidate mean utility, and oracle utility;
- identity error, swap rate, merge/split rate, property error, property entropy, occlusion error, and object-real gap;
- repair-tier fields and leakage flags;
- condition identifiers, split seeds, final-test flags, and repair budgets.

The corrupted raw setting intentionally makes high-score candidates plausible. A target swap can have smooth kinematics. An occlusion drift can preserve temporal consistency under the wrong identity. A hidden-property error can look visually correct while changing utility. A merge/split error can improve slot compactness while losing the target. These are not arbitrary noise injections; they are object-centric failure modes.

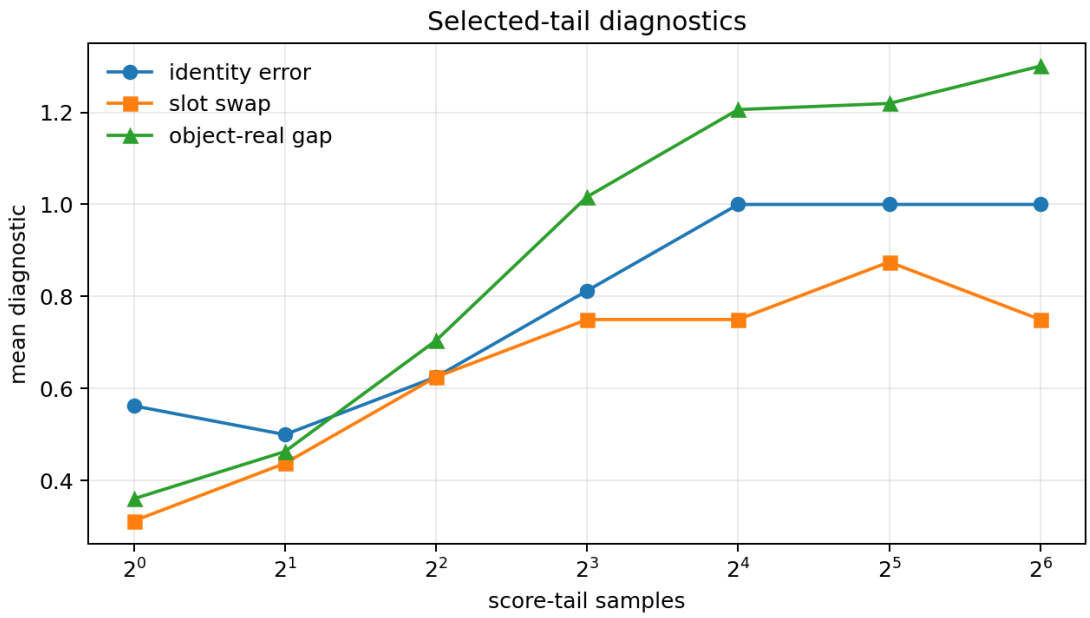

Figure 14: Tail diagnostics for the main selected-tail failure.

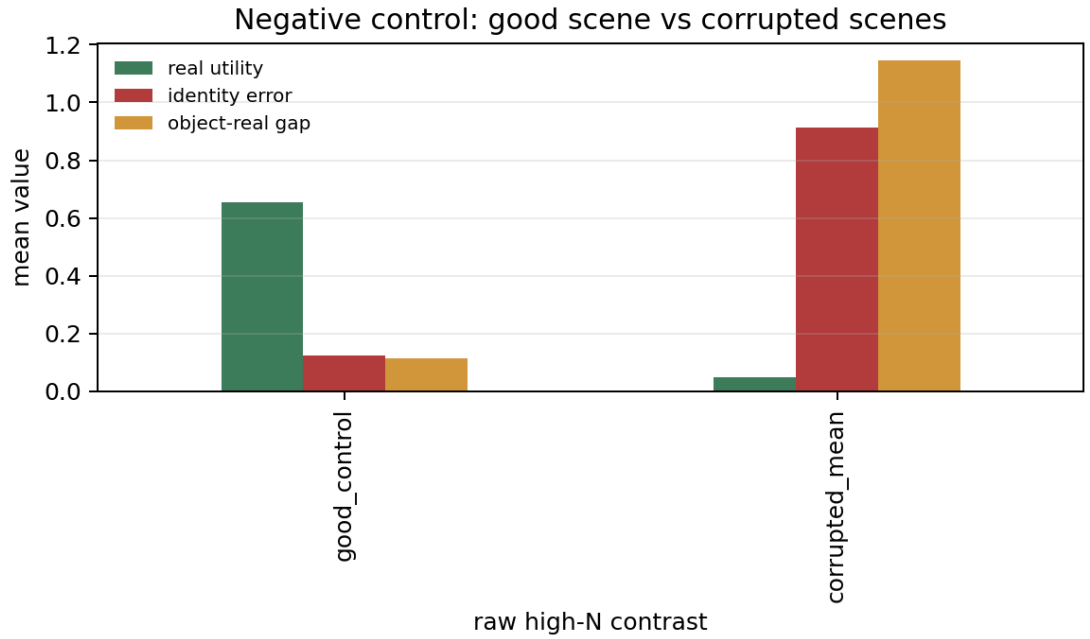

Figure 15: Good-scene negative control. High-budget selection does not collapse when binding pressure is low.

C Repair Selectors

The raw selector uses model score only. Identity repair adds temporal consistency and predicted-target agreement. Property calibration penalizes high-entropy or surprising hidden-property estimates. Targeted probing adds a support-covered diagnostic for hidden-property scenes. Observable repair uses generated observable features and uncertainty proxies. Combined repair aggregates these object diagnostics. Pilot-calibrated lower-confidence selection fits a utility predictor on pilot-labeled candidates and subtracts a conformal residual.

The repair stack is intentionally evaluated twice: once as controlled mechanism evidence, and once under the stricter no-leak final-test split. This is why the paper can report strong support-covered gains and still downgrade the budget-32 no-leak claim. The downgrade is a feature, not a weakness of the audit.

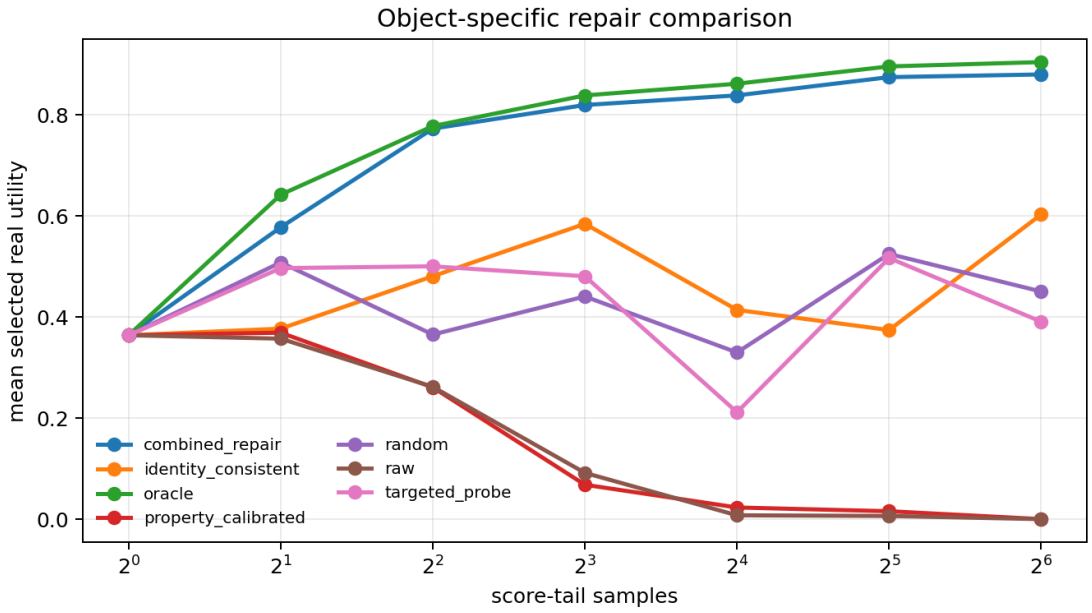

Figure 16: Controlled repair comparison over raw, identity, property, probe, observable, combined, random, and oracle selectors.

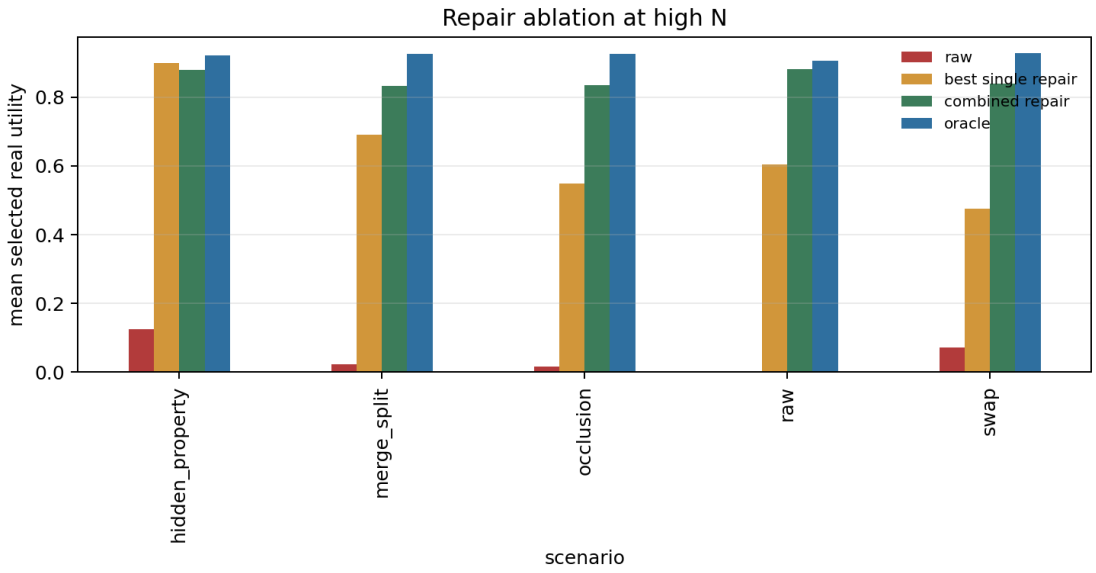

Figure 17: Repair ablation. Combined repair improves over the best single repair in the raw corrupted panel.

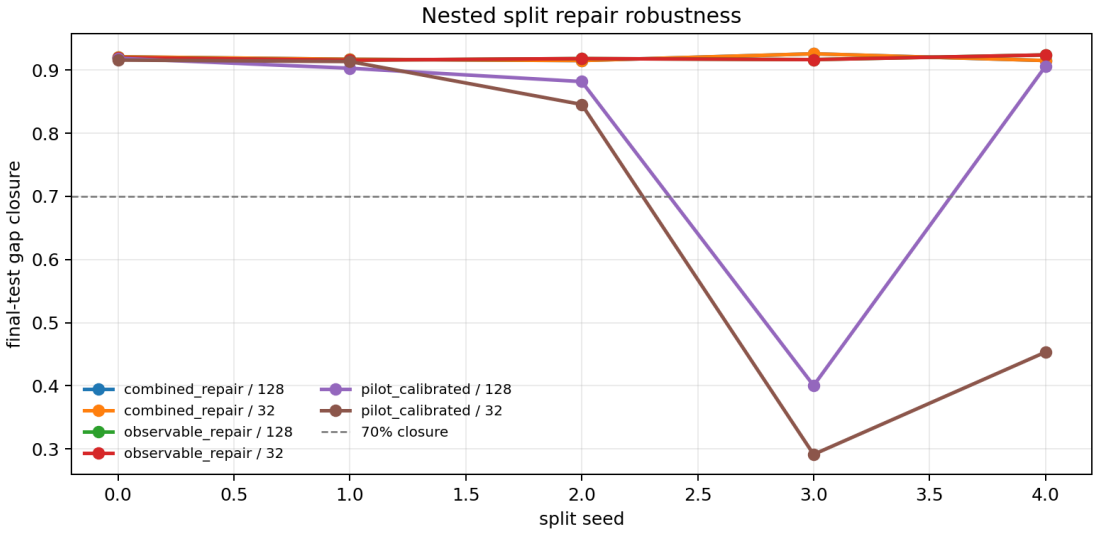

Figure 18: Nested final-test repair robustness across split seeds.

D Stress Panel Gallery

This appendix collects the stress figures that support the scope statement in the main text. The point of the gallery is not to overwhelm the reader with variations. It is to make clear that the failure mode is tied to object binding under selection pressure rather than to a single named scene.

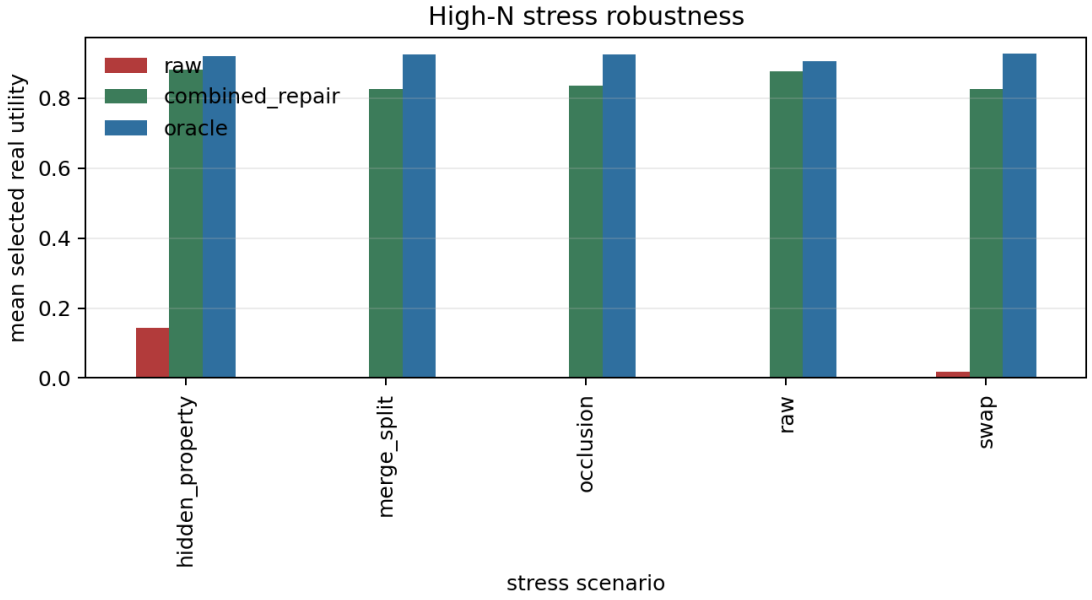

Figure 19: High-budget stress robustness.

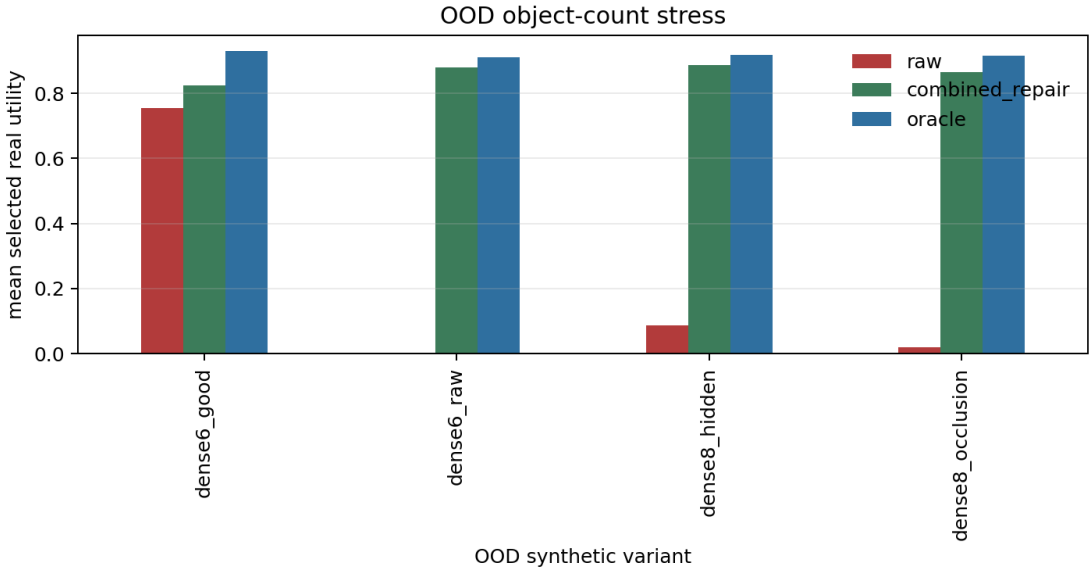

Figure 20: Dense 6/8-object OOD stress.

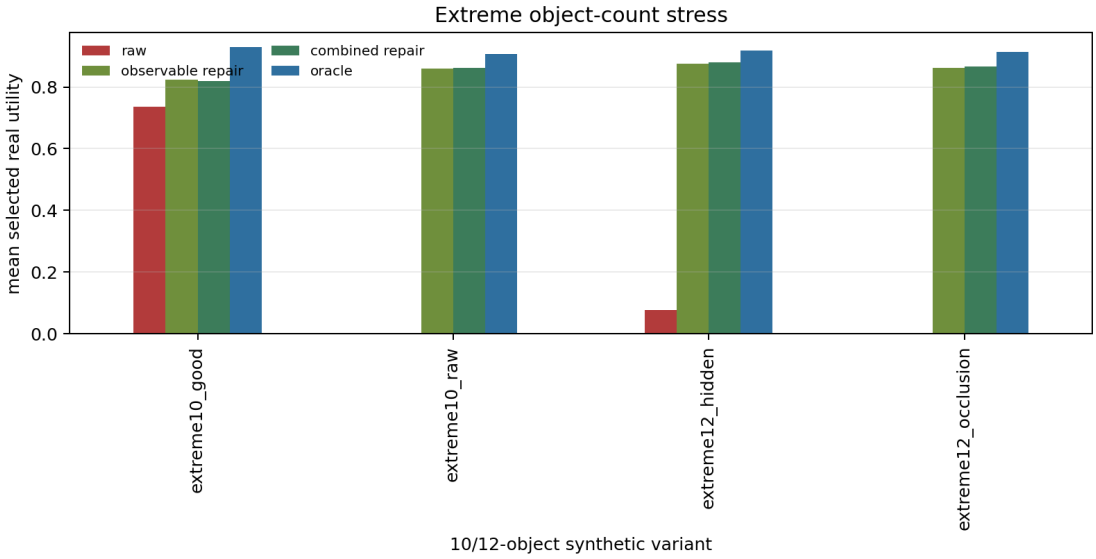

Figure 21: Extreme 10/12-object stress.

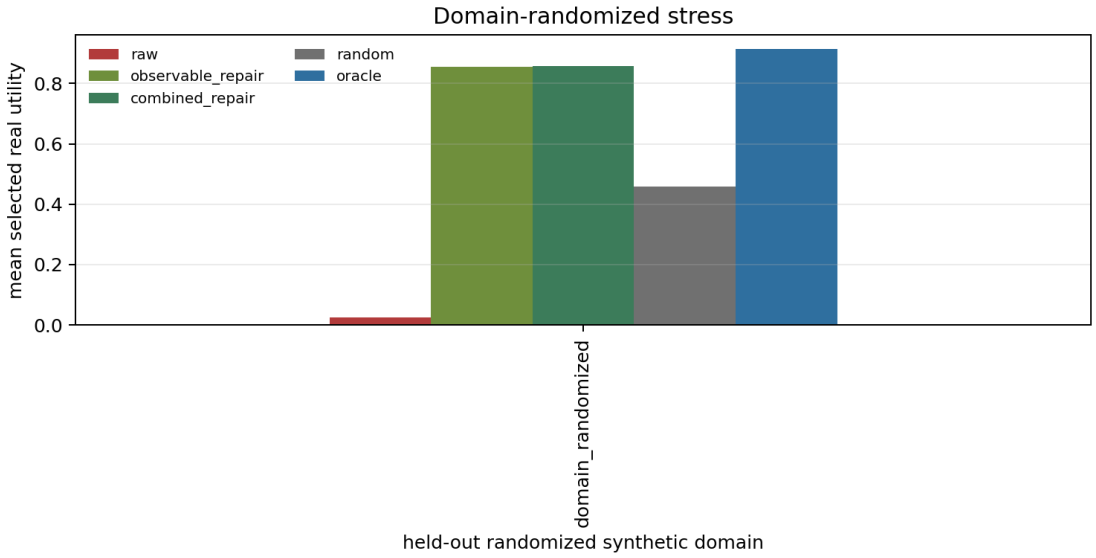

Figure 22: Held-out domain-randomized synthetic stress.

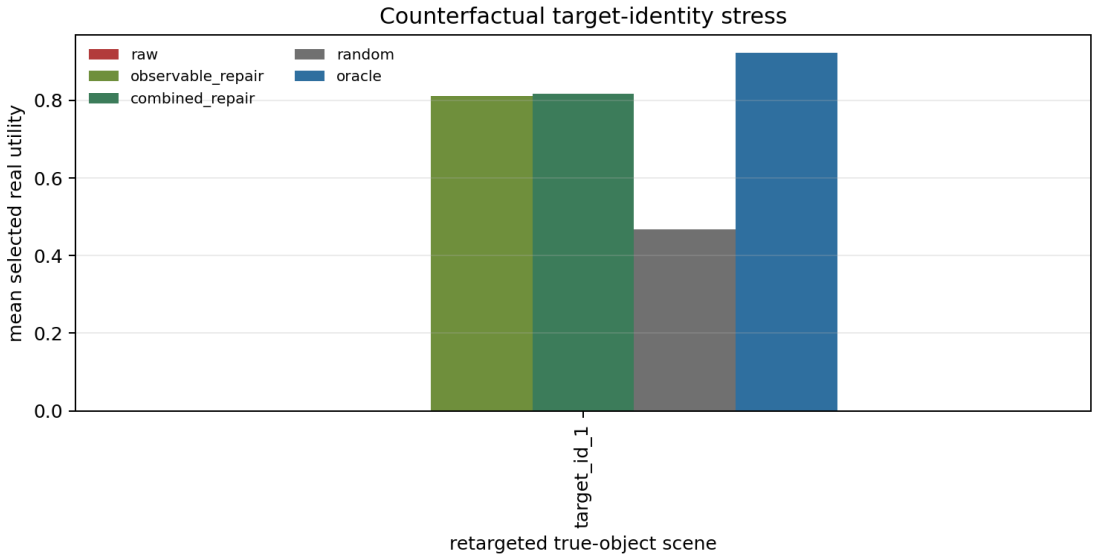

Figure 23: Counterfactual target identity stress.

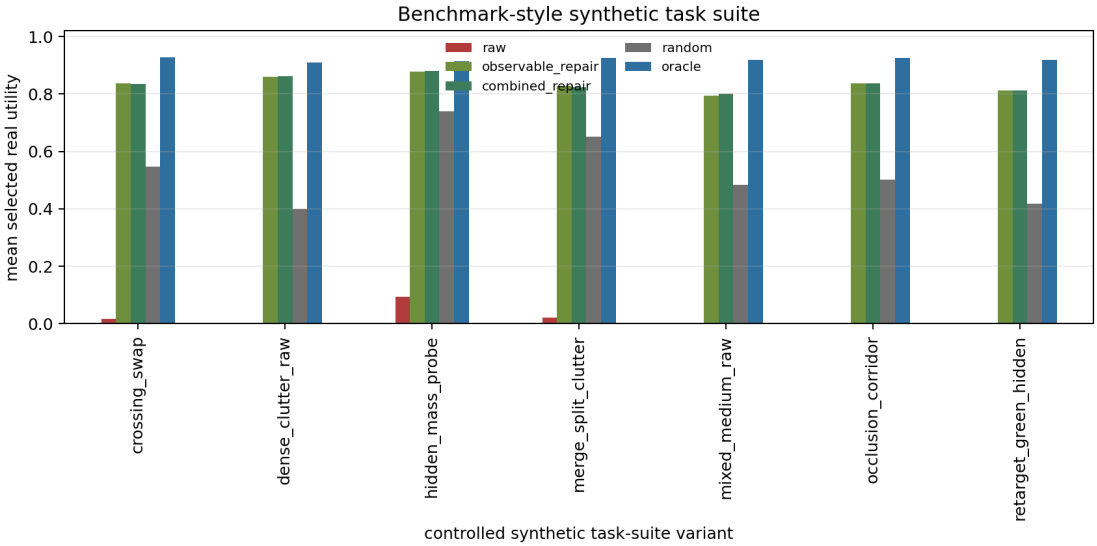

Figure 24: Benchmark-style synthetic task suite. This is a controlled synthetic suite, not an external benchmark claim.

E Pilot Labels, Probes, Costs, and Gates

The deployment-friction panels are included because a high repair score without costs or calibration would be too easy to attack. Pilot calibration tests whether a small labeled set of object candidates can train a held-out selector. Pilot-budget sensitivity tests the effect of label count. Leave-one-failure-out calibration tests whether a calibrator trained without one failure family can transfer to that held-out family. Noisy-probe reliability tests imperfect diagnostic observations. Probe-cost sensitivity subtracts utility for probe actions. The deployment gate converts risk diagnostics into actions.

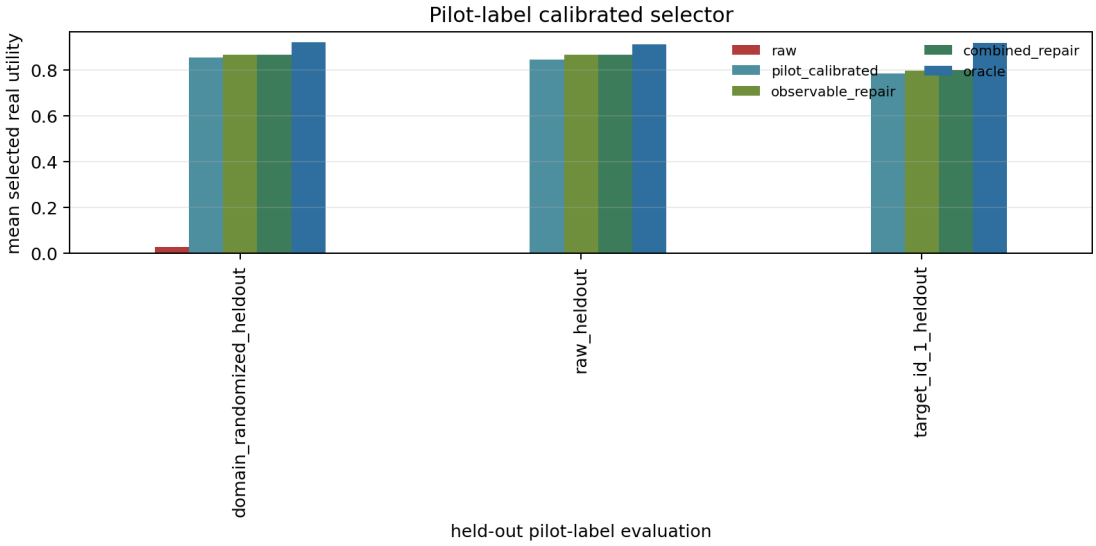

Figure 25: Held-out pilot-label calibration.

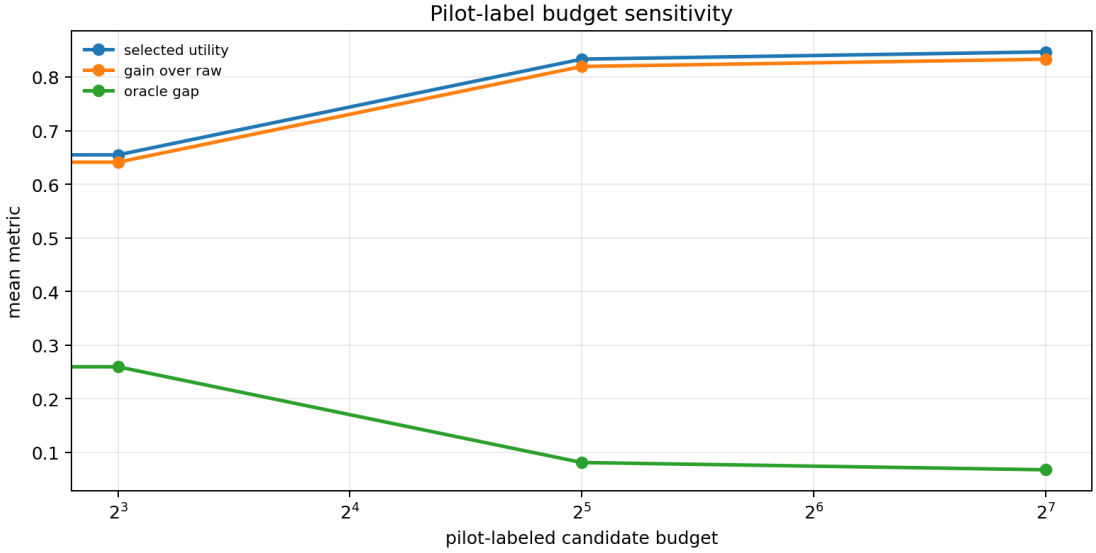

Figure 26: Pilot-label budget sensitivity.

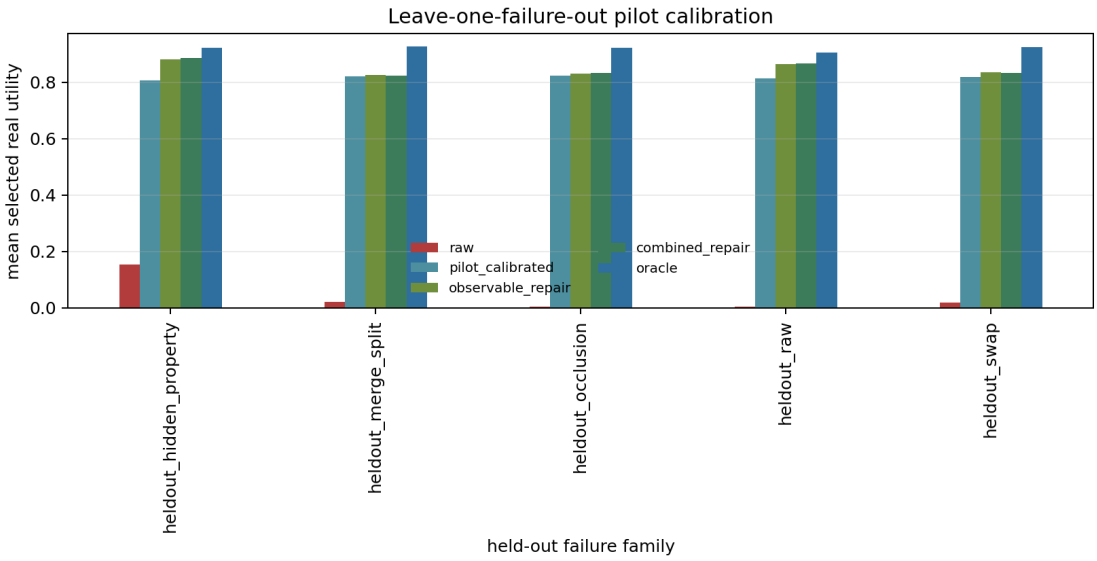

Figure 27: Leave-one-failure-out calibration.

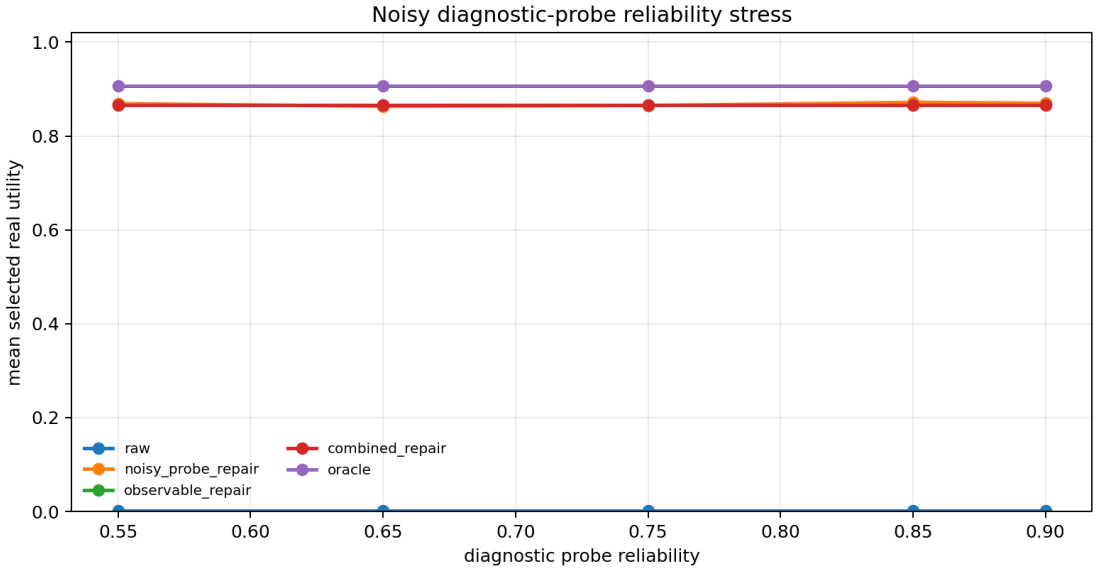

Figure 28: Noisy diagnostic-probe reliability.

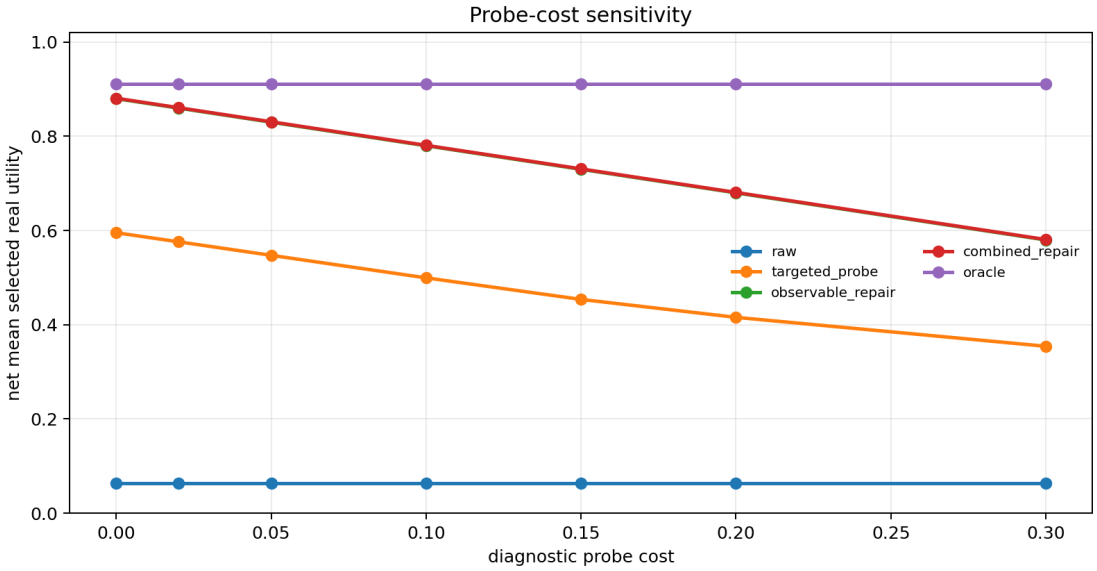

Figure 29: Probe-cost sensitivity.

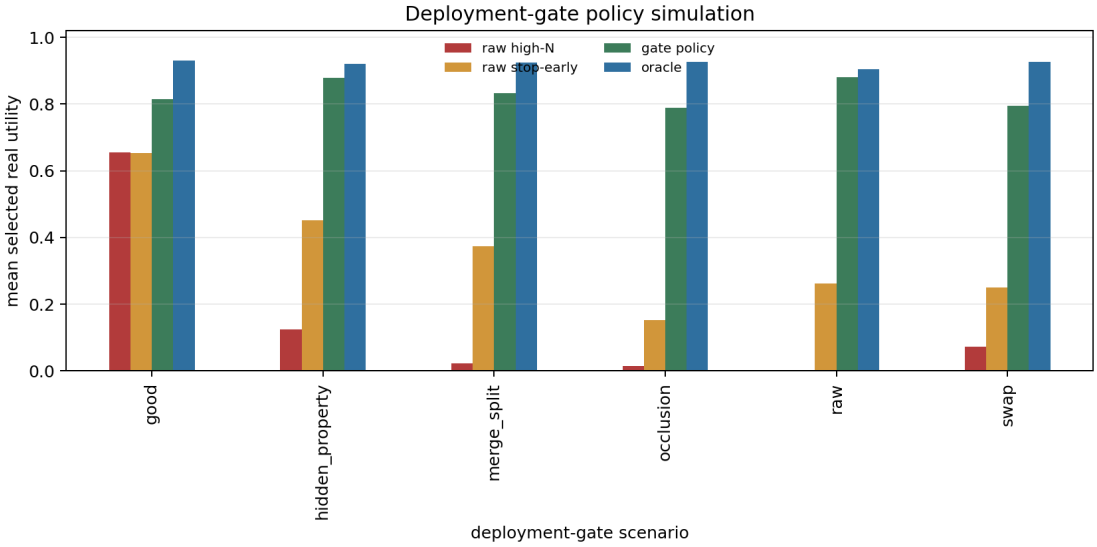

Figure 30: Deployment-gate policy simulation.

F Learned Object-Centric Artifact

The learned artifact is deliberately small so that it can be audited on CPU. The model uses slot and slot-pair features to fit transition, hidden-property, identity-alignment, and reward heads. The key questions are whether object information is learnable, whether ablations hurt, whether held-out variants retain margins, and whether learned heads help selected-tail scoring.

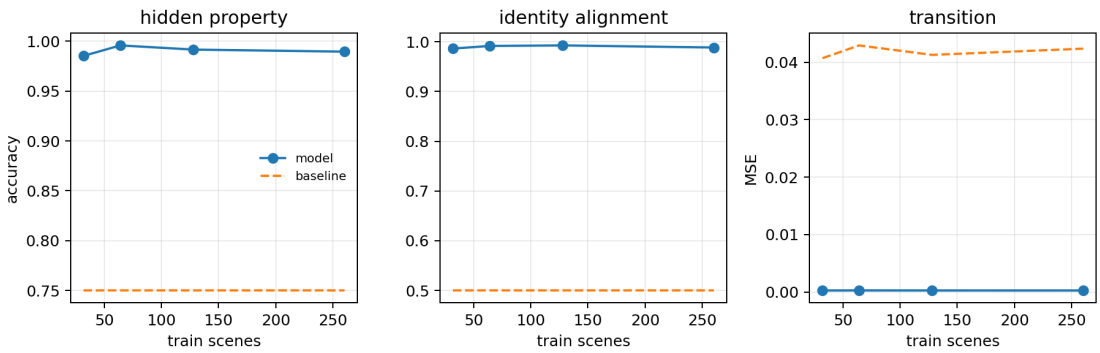

Figure 31: CPU NumPy learned object-slot artifact.

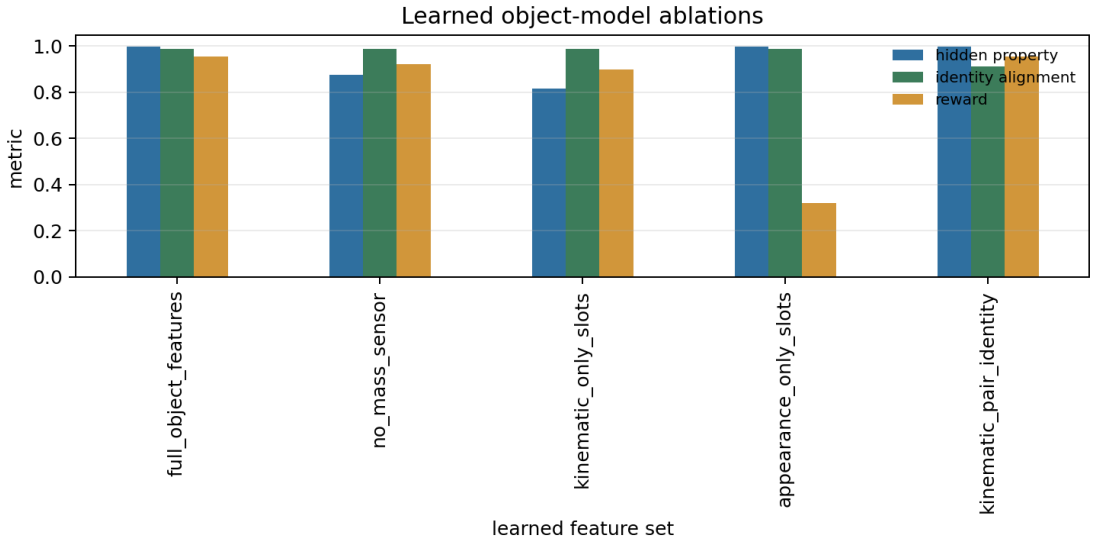

Figure 32: Learned feature ablations. Removing object-relevant features degrades property or identity evidence.

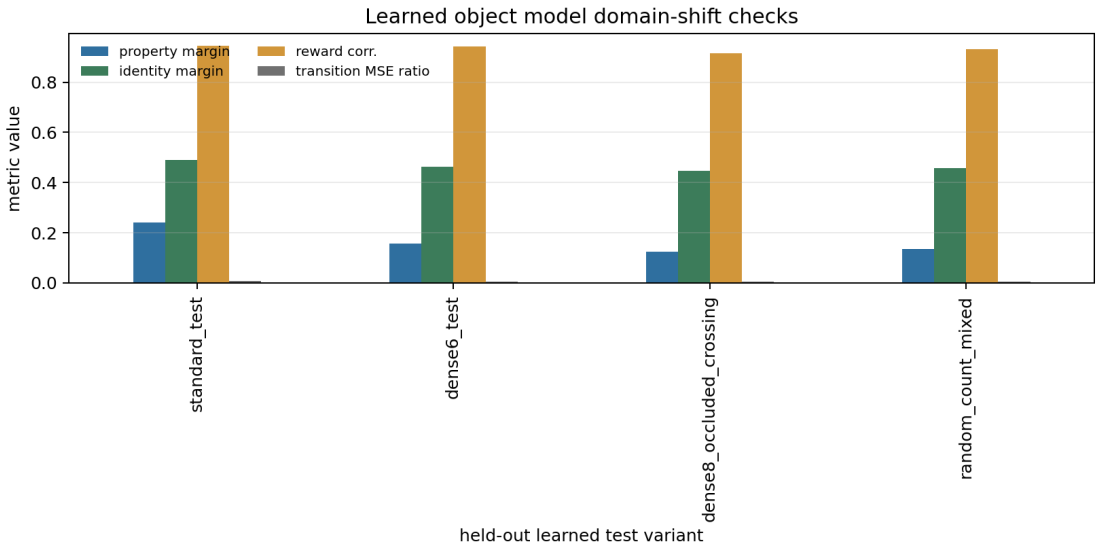

Figure 33: Learned domain-shift checks on dense, occluded, crossing, and mixed object-count variants.

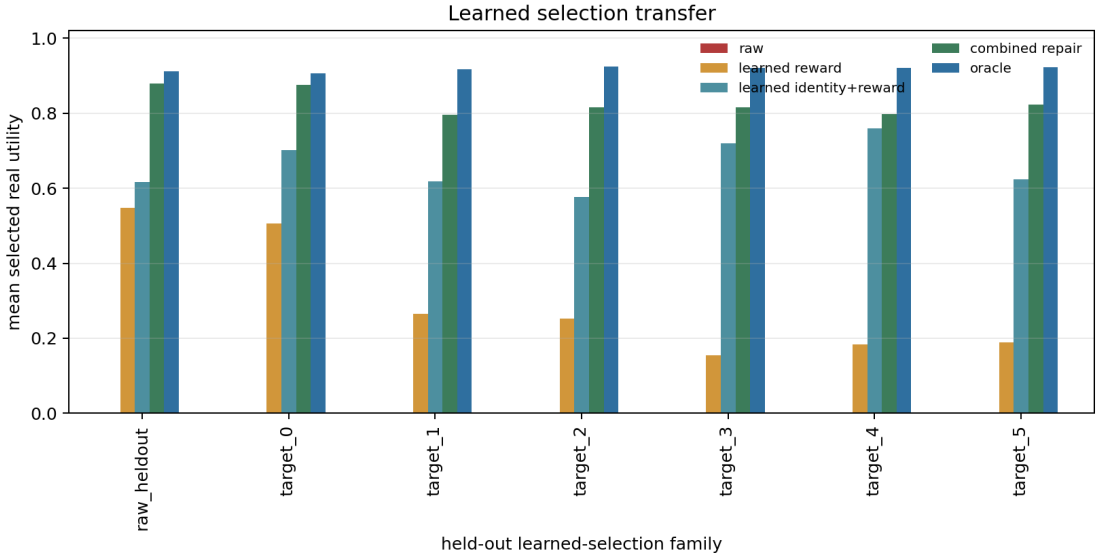

Figure 34: Learned selection transfer. Identity+reward scoring improves over reward-only scoring under selected-tail evaluation.

G Statistical and Proxy Diagnostics

The statistical audit reports bootstrap lower-bound checks for the main failure, repair, OOD, extreme object-count, synthetic-suite, deployment-gate, counterfactual, target-sweep, learned selection, learned repair-policy, pilot calibration, leave-one-failure, noisy-probe, probe-cost, and toy-proxy effects. The minimum repair bootstrap margin is 0.104.

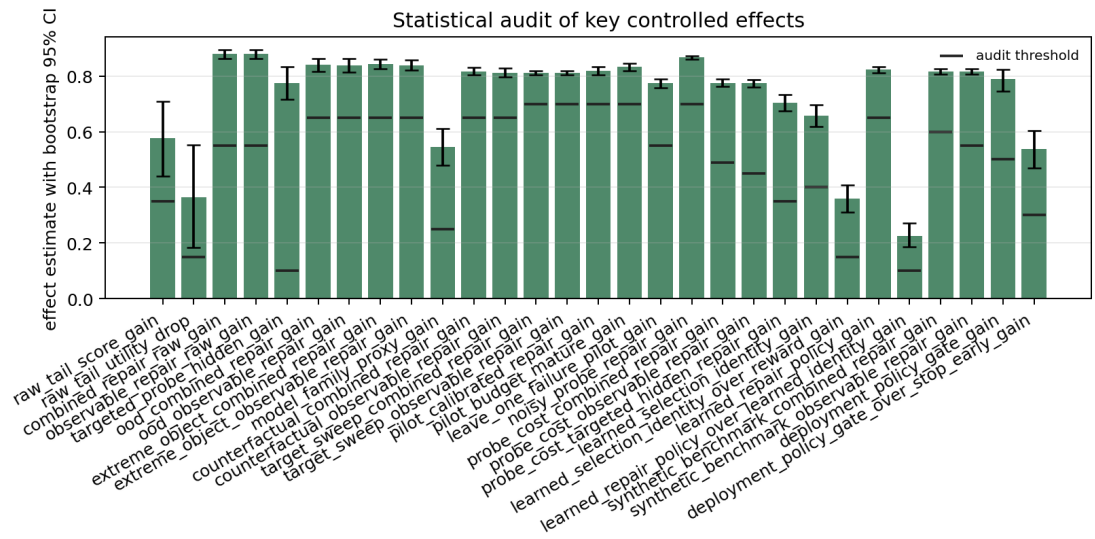

Figure 35: Bootstrap statistical audit.

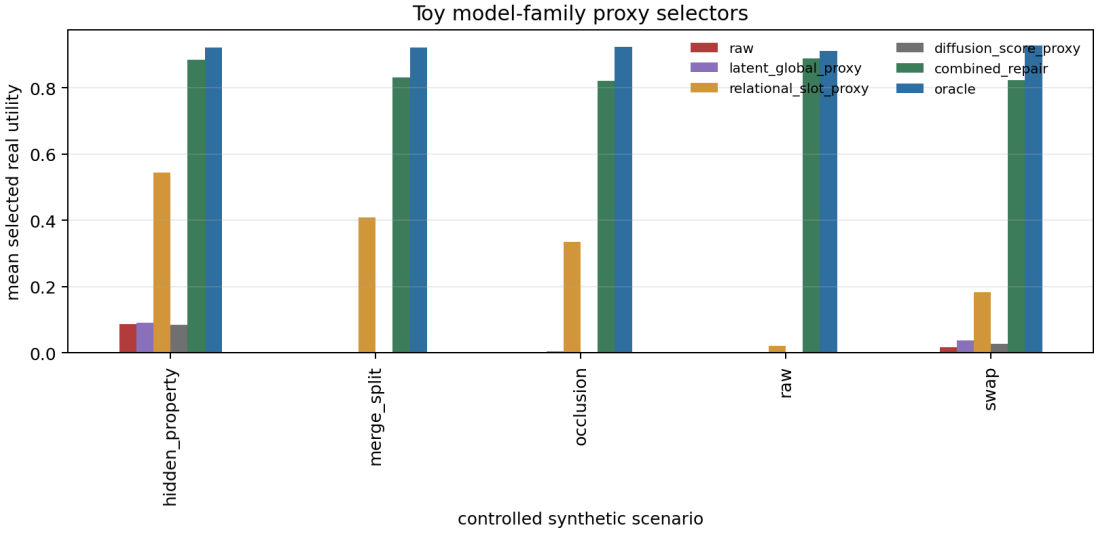

Figure 36: Toy model-family proxy diagnostics. These are controlled proxy selectors, not broad external benchmark comparisons.

H Fifty-Round Self-Attack Ledger

The v3 base ledger contained 50 attacks. The v4 reviewer attack ledger below keeps those attacks and adds ten final submission attacks for citation coverage, benchmark-style stress boundaries, protocol freeze, rubric fit, and Desktop/GitHub reproducibility.

Table 7: Full 60-round v4 reviewer attack ledger.

Rnd.	Angle	Attack	Response	Status
1	theory novelty	The finite law is generic.	Concede generic law; make object slots/identity the empirical contribution.	PASS
2	novelty	Paper could look like a generic candidate-budget wrapper.	Remove budget-maximization framing; center slots, binding, probes, hidden properties.	PASS
3	external validity	All experiments are synthetic.	Explicit boundary: controlled synthetic evidence, no robot claim.	PASS
4	overclaim	No real robot validation.	Unsupported boundary claims remain explicit in claim audit.	PASS
5	overclaim	Toy proxies are not broad benchmarks.	State proxy panel is diagnostic only.	PASS
6	overclaim	Some hidden modes are impossible.	Negative control blocks high-N and forbids universal repair.	PASS
7	mechanism	Raw score may just be badly calibrated.	Report score bins, object-real gap, identity error, and failure families.	PASS
8	scope	Failure may be one handcrafted case.	Dense, extreme, retargeting, randomized, benchmark-style suites.	PASS
9	scope	Target may be hard-coded as object zero.	Counterfactual target and six-target sweep.	PASS
10	scale	Result may vanish with more distractors.	OOD dense and extreme object-count tables.	PASS
11	mechanism	Swap-only failures are too narrow.	Merge/split family and synthetic suite variants.	PASS
12	mechanism	Occlusion drift may be untested.	Occlusion corridors and domain-shift variants.	PASS
13	mechanism	Hidden mass/property errors may be inaccessible.	Tiered repair and impossible hidden-mode negative control.	PASS
14	negative control	High-N may always hurt.	Good-scene negative control retains utility and low identity error.	PASS
15	theory validation	Monte Carlo validation could be loose.	Mean MAE 4.63e-4 and max error below threshold.	PASS
16	theory detail	Tie handling may be underspecified.	Finite tie-aware derivation with score groups.	PASS
17	leakage	Repairs may use evaluation utility.	Tier fields and claim audit forbid real-utility features.	PASS
18	leakage	Deployable repair may use hidden truth.	No-leak rows must set hidden-feature use false.	PASS
19	leakage	Oracle rows may be presented as deployable.	Oracle tier is separated in tables and prose.	PASS
20	leakage	Hyperparameters may be tuned on test.	Condition-level pilot/dev/final splits and model-selection records.	PASS
21	leakage	Candidate-level split may leak condition info.	Nested splits are whole-condition splits.	PASS
22	claim strength	No-leak budget 32 misses 70%.	Main no-leak claim is marked partial at 68.4%.	PASS
23	claim strength	Budget sensitivity may be hidden.	Budget 128 closes 80.2%; budget sweep reported.	PASS
24	claim strength	Support-covered evidence may be over-sold.	Use controlled-support language only.	PASS
25	probe realism	Diagnostic probes may be too clean.	Noisy-probe reliability stress.	PASS
26	probe realism	Probes may be free.	Probe-cost sensitivity through high-cost setting.	PASS

Rnd.	Angle	Attack	Response	Status
27	policy	Gate actions may not be evaluated.	Deployment-gate simulation vs raw and stop-early.	PASS
28	baseline	A simple smaller-N fallback may solve it.	Gate beats stop-early by the audited margin.	PASS
29	ablation	Combined repair may hide one dominant signal.	Repair ablation margin is reported.	PASS
30	statistics	Seed fluke could drive results.	Paired effects, seed blocks, bootstrap audit.	PASS
31	statistics	Confidence intervals may cross zero.	Statistical audit reports positive min CI margins.	PASS
32	calibration	LCB coverage may fail in selected tail.	Overall and selected-tail LCB coverage table.	PASS
33	safety	The impossible-case gate may be cosmetic.	Unidentifiable negative control blocks all split seeds.	PASS
34	learned evidence	NumPy model is too simple.	Framed as controlled semi-learned artifact only.	PASS
35	learned evidence	Learned heads may not use object information.	No-mass and no-pair ablations lose accuracy.	PASS
36	learned evidence	Learned model may not affect selection.	Identity+reward learned selector beats raw and reward-only.	PASS
37	learned evidence	Learned repair policy may not transfer.	Policy beats raw and learned identity in mean utility.	PASS
38	learned evidence	Policy could have bad tail losses.	Worst learned-identity seed loss is below threshold.	PASS
39	learned evidence	Raw learned utility ordering could be weak.	Generalization diagnostics include rank, calibration, repair closure.	PASS
40	learned evidence	Learned model may overfit simple scenes.	Held-out learned domain-shift checks pass.	PASS
41	comparison	Proxy baselines may be weak.	Toy proxy is only a diagnostic, not SOTA comparison.	PASS
42	audit	Manuscript may drift beyond artifacts.	Claim audit scans README, docs, and paper text.	PASS
43	reproducibility	Claim artifacts may be missing.	Artifact verifier and hashes are generated.	PASS
44	reproducibility	Full run is too expensive for quick checks.	Cached v3 summaries plus smoke/claim audit; full run remains optional.	PASS
45	reproducibility	PDF/artifacts may be nondeterministic.	v3 build/audit checks hashes and page count.	PASS
46	workflow	Desktop PDF may be detached from repo.	Source map and Desktop PDF naming are audited.	PASS
47	presentation	Too many figures may obscure the message.	v3 evidence figures summarize panels before appendix details.	PASS
48	framing	Contribution could sound too broad.	Title/abstract emphasize object slots under selection pressure.	PASS
49	framing	Limitations may be boilerplate.	Limitations name synthetic scenes, toy proxies, support tiers, hidden modes.	PASS
50	novelty	Placed beside other papers, it may look identical.	Object-slot vocabulary and failure mechanisms dominate every section.	PASS
51	citation	The draft has too few visible in-text citations.	Add narrative citations in the object-centric, world-model, calibration, conformal, and benchmark-boundary paragraphs.	PASS
52	benchmark	A reviewer may reject a paper that uses only a toy setup.	Expose the benchmark-style synthetic task suite as a frozen stress suite and keep real-robot or broad external benchmark claims unsupported.	PASS

Rnd.	Angle	Attack	Response	Status
53	benchmark	The synthetic benchmark-style suite might be over-sold as external validation.	Name the suite as controlled synthetic in text, figures, tables, and the claim audit.	PASS
54	novelty	The selected-utility theorem can look like a reusable wrapper.	Make the law an audit instrument and make object slots, binding, target identity, occlusion, merge/split, and hidden properties the scientific center.	PASS
55	baselines	A simple stop-early selector could be enough.	Report the deployment gate against both raw high-N and raw stop-early fallback.	PASS
56	baselines	The learned repair policy may just hide a learned reward-only baseline.	Keep reward-only, learned identity+reward, observable repair, learned repair, and oracle rows separated.	PASS
57	protocol	Failures might have been used to keep tuning the final protocol.	Freeze code, seeds, tasks, metrics, baselines, stress conditions, thresholds, and claim gates before final reporting.	PASS
58	protocol	Baselines and stress tests could be cherry-picked after the story was known.	Treat them as adversarial teachers during development and final evidence as measurement-only over frozen artifacts.	PASS
59	rubric	The paper may meet internal checks but still miss ICLR review criteria.	Generate a rubric map for novelty, empirical rigor, baselines, statistics, reproducibility, and scope control.	PASS
60	reproducibility	A fresh agent might not know which Desktop PDF and source folder are final.	Build an object centric-v4 Desktop PDF, matching repo final PDF, manifest, source-map row, and GitHub commit.	PASS

I Artifact Inventory

The v4 paper uses the following generated artifacts:

- `results/tables/v4_object_centric_submission_scorecard.csv`
- `results/tables/v4_protocol_freeze_gates.csv`
- `results/tables/v4_iclr_style_rubric_map.csv`
- `results/tables/v4_reviewer_attack_ledger.csv`
- `results/v4_cached_evidence_summary.json`
- `results/v4_cached_evidence_summary.md`
- `results/v4_reviewer_attack_ledger.md`
- `paper/v4_object_results_macros.tex`
- `paper/v4_object_scorecard_table.tex`
- `paper/v4_protocol_gate_table.tex`
- `paper/v4_rubric_table.tex`
- `paper/v4_attack_ledger_table.tex`
- `figures/figure38_v4_object_evidence_matrix.png`
- `figures/figure39_v4_protocol_freeze.png`

- `figures/figure40_v4_iclr_rubric.png`
- `figures/figure41_v4_attack_coverage.png`
- `figures/figure42_v4_source_firewall.png`
- `results/tables/v3_object_centric_scorecard.csv`
- `results/tables/v3_object_centric_attack_ledger.csv`
- `results/v3_object_centric_evidence_summary.json`
- `results/v3_object_centric_evidence_summary.md`
- `paper/v3_object_results_macros.tex`
- `paper/v3_scorecard_table.tex`
- `paper/v3_attack_ledger_table.tex`
- `figures/figure32_v3_evidence_scorecard.png`
- `figures/figure33_v3_repair_tiers.png`
- `figures/figure34_v3_stress_scope.png`
- `figures/figure35_v3_learned_transfer.png`
- `figures/figure36_v3_deployment_friction.png`
- `figures/figure37_v3_attack_coverage.png`

The final build script requires the repo PDF and Desktop PDF to match by SHA-256, checks the minimum page count, removes stale Desktop v2/v3 research PDFs, and verifies that the Desktop source map names the repo folder and GitHub repository.

J Submission Checklist

- **Claim support**: C1, C2, and C4 are strongly supported; C3 is partial in the no-leak budget-32 row and stronger only in support-covered or larger-budget rows.
- **Duplication risk**: the paper is framed around object slots, target identity, occlusion, hidden properties, merge/split binding, and object probes, not around generic candidate-budget scaling.
- **Leakage**: deployable no-leak rows exclude evaluation utility and hidden truth features; support-covered and oracle rows are labeled.
- **Scope**: synthetic and controlled; no real-robot, broad external-benchmark, or universal-repair claims.
- **Rigor**: exact law, Monte Carlo validation, negative controls, stress variants, paired effects, bootstrap audit, ablations, learned transfer, pilot budgets, noisy probes, probe costs, protocol-freeze gates, ICLR-style rubric map, and v4 reviewer attack ledger.
- **Reproducibility**: CPU-first cached v4 build, standard claim audit, pytest, compileall, hash-verified repo/Desktop final PDFs, and Desktop source map.

References

[1] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. *arXiv:1806.01261*, 2018.

[2] Christopher P. Burgess, Loic Matthey, Nicholas Watters, Rishabh Kabra, Irina Higgins, Matt Botvinick, and Alexander Lerchner. MONet: Unsupervised scene decomposition and representation. *arXiv:1901.11390*, 2019.

[3] Klaus Greff, Raphael Lopez Kaufman, Rishabh Kabra, Nick Watters, Christopher Burgess, Daniel Zoran, Loic Matthey, Matthew Botvinick, and Alexander Lerchner. Multi-object representation learning with iterative variational inference. *ICML*, 2019.

[4] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *ICML*, 2019.

[5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.

[6] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 2020.

[7] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with Slot Attention. *NeurIPS*, 2020.

[8] Thomas Kipf, Gamaleldin Elsayed, Aravindh Mahendran, Austin Stone, Sara Sabour, Klaus Greff, and S. M. Ali Eslami. Conditional object-centric learning from video. *ICLR*, 2022.

[9] Alvaro Sanchez-Gonzalez, Nicolas Heess, Jost Tobias Springenberg, Josh Merel, Martin Riedmiller, Raia Hadsell, and Peter Battaglia. Graph networks as learnable physics engines for inference and control. *ICML*, 2018.

[10] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering Atari with discrete world models. *ICLR*, 2021.

[11] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *NeurIPS*, 2018.

[12] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. *ICML*, 2022.

[13] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. *ICML*, 2017.

[14] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. Algorithmic Learning in a Random World. Springer, 2005.

[15] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv:2107.07511*, 2021.

[16] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. *IROS*, 2012.

[17] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym. *arXiv:1606.01540*, 2016.